

Binary Fields and Field Extensions

Theory and Implementation in Rust

Paolo De Marinis

September 20, 2026

Contents

Introduction	3
I Binary Representation and Polynomial Arithmetic	4
0 Binary Coordinates and Word Representation	5
0.1 Algebra	5
0.2 Representation	5
0.3 Implementation	7
0.4 Verification	7
1 Binary Extensions: Degree, Basis, and a First Model	8
1.1 Algebra	8
1.2 Representation	12
1.3 Implementation	13
1.4 Verification	13
2 Formal Polynomials and Binary Encoding	14
2.1 Algebra	14
2.2 Representation	15
2.3 Implementation	16
2.4 Verification	16
3 Carry-Less Polynomial Multiplication	17
3.1 Algebra	17
3.2 Representation	19
3.3 Implementation	20
3.4 Verification	21
II Arithmetic in $\mathbb{F}_{2^{128}}$	22
4 Field Construction and Specialized Reduction	23
4.1 Algebra	23
4.2 Representation	26
4.3 Implementation	28
4.4 Verification	28
5 The Gf128 Type and Field Operators	30
5.1 Algebra	30
5.2 Representation	30
5.3 Implementation	31
5.4 Verification	33
6 Squaring and Inversion	34
6.1 Squaring	34
6.2 Inversion	41
7 Constant-Time Arithmetic	46
7.1 The constant-time requirement	46
7.2 Audit of the existing arithmetic	47
7.3 Masks and branchless selection	47
7.4 Constant-time carry-less multiplication	48
7.5 Sensitive predicates	50
7.6 Inversion without a data-dependent exit	51
7.7 Limits of the guarantee	53
Next steps	54
References	55

Introduction

Scope and current status

These notes are written as a study guide and closely follow my actual learning and implementation process. Rust concepts are introduced when they become necessary to translate the algebra into code, while exercises, implementation tasks, and verification sections reconstruct the steps followed during development. The aim is therefore not only to document the final implementation, but also to preserve the reasoning that led to it.

These notes develop the portable arithmetic of a concrete model of $\mathbb{F}_{2^{128}}$, currently implemented in the `binary_ffft` library, deriving algorithms from the algebra and the word representation. Chapters 0–3 develop binary coordinates, a first model of $\mathbb{F}_4$, formal polynomials, and carry-less multiplication. Chapters 4–6 construct the field as a polynomial quotient, develop specialized reduction, and introduce the `Gf128` type through squaring and Itoh–Tsuji inversion. Chapter 7 introduces a constant-time discipline at the source level, while the preceding chapters preserve the intermediate stages of the study process. Cantor bases and additive transforms, including the Dyadic AFFT, are the next stage and are not yet implemented. Each development connects algebra, representation, implementation, and verification.

Current repository structure

The following tree corresponds to commit `df2c644` of `binary_ffft`.

```
binary_ffft/  
|-- Cargo.toml  
|-- README.md  
|-- src/  
|   |-- lib.rs  
|   |-- bits.rs  
|   |-- polynomial.rs  
|   |-- field/  
|       |-- mod.rs  
|       |-- gf128.rs  
|       |-- portable.rs  
|-- tests/  
|   |-- common/  
|   |   |-- slow_clmul.rs  
|   |   |-- slow_reduce.rs  
|   |-- bits.rs  
|   |-- polynomial.rs  
|   |-- field_portable.rs  
|   |-- gf128.rs  
|-- examples/  
|   |-- gf4.rs  
|   |-- word.rs
```

The public arithmetic type is `binary_ffft::field::Gf128`: its operations interpret words as field elements. The module `field::portable` contains the only current backend, which operates on words. The functions `mul_p128` and `square_p128` return reduced coordinates, while `clmul128` and `spread128` produce the unreduced wide pair `(lo, hi)`.

The connection is direct: `gf128.rs` calls the functions in `portable`. The primitives remain public for comparisons in tests and for future benchmarks, while higher-level algorithms will use the `Gf128` interface. Tests are organized separately by component.

The implementations are intended for study and provide no guarantee of constant-time execution. A fixed operation count in a formula alone does not establish the timing behavior of compiled code.

Part I

Binary Representation and Polynomial Arithmetic

Chapter 0

Binary Coordinates and Word Representation

The `bits.rs` module provides operations for coordinate access, updates, and conversion.

0.1 Algebra

A *field* is a structure $(K, +, \cdot, 0, 1)$ with $0 \neq 1$ such that $(K, +)$ is an abelian group, $(K \setminus \{0\}, \cdot)$ is an abelian group, and multiplication distributes over addition. In particular, $ab = 0$ implies $a = 0$ or $b = 0$.

Our base field is

$$\mathbb{F}_2 = \{0, 1\}, \quad a + b = a \oplus b, \quad ab = a \wedge b$$

for $a, b \in \{0, 1\}$. For $m \geq 1$, set

$$V_m := \mathbb{F}_2^m = \{(a_0, \dots, a_{m-1}) : a_i \in \mathbb{F}_2\},$$

with coordinatewise addition and scalar multiplication. In this section, V_m is only a vector space over $\mathbb{F}_2$, and words represent its coordinates. Defining a field structure on these coordinates requires additional information, introduced in the following sections.

0.2 Representation

0.2.1 Coordinates and word encoding

For $1 \leq m \leq 64$, define

$$\text{pack}_m(a_0, \dots, a_{m-1}) := \sum_{i=0}^{m-1} \tilde{a}_i 2^i,$$

where $\tilde{a}_i \in \{0, 1\} \subset \mathbb{Z}$ represents a_i . The codomain is $\{0, \dots, 2^m - 1\}$, viewed as the set of canonical words of width m .

The inverse map is

$$\text{unpack}_m(x) := \left(\left\lfloor \frac{x}{2^0} \right\rfloor \bmod 2, \dots, \left\lfloor \frac{x}{2^{m-1}} \right\rfloor \bmod 2 \right), \quad 0 \leq x < 2^m.$$

Thus

$$\text{unpack}_m(\text{pack}_m(a)) = a, \quad \text{pack}_m(\text{unpack}_m(x)) = x.$$

Addition in V_m corresponds to XOR on words:

$$\text{pack}_m(a + b) = \text{pack}_m(a) \oplus \text{pack}_m(b).$$

We introduce no separate function for this addition: on the storage type, its implementation is exactly the Rust operator `^`. Coordinatewise AND, integer multiplication, carry-less multiplication, and multiplication in the field to be constructed as a polynomial quotient are different operations.

Example 0.1 (Encoding the word 13). Since $13 = 2^0 + 2^2 + 2^3$, we have

$$\text{unpack}_4(13) = (1, 0, 1, 1).$$

As a 64-bit word, the same encoding represents $(1, 0, 1, 1, 0, \dots, 0) \in V_{64}$.

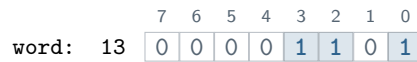


Figure 0.1: Eight bits of the word are shown, with bit 0 on the right. Bits 63 through 8 are all zero. The coordinate tuple, in contrast, is read in increasing index order.

0.2.2 Rust bitwise operators and mathematical notation

The table compares mathematical notation with Rust syntax, with shifts on unsigned `u64` integers.

Rust	Math	Operation	Effect on each bit
<code>&</code>	$\wedge$	bitwise AND	equals 1 only when both bits are 1.
<code> </code>	$\vee$	bitwise OR	equals 1 when at least one bit is 1.
<code>^</code>	$\oplus$	bitwise XOR	equals 1 when the bits differ.
<code>!</code>	$\neg$	bitwise NOT	exchanges 0 and 1 at every position.
<code><<</code>	$x \ll s$	left shift	moves bits toward higher indices and inserts zeros on the right.
<code>>></code>	$x \gg s$	right shift	moves bits toward lower indices and inserts zeros on the left.

Example 0.2 (Bitwise operators on two words). For $x = 0b1010$ and $y = 0b1100$, the Rust operations $x \& y$, $x | y$, and $x ^ y$ return `0b1000`, `0b1110`, and `0b0110`, respectively. The operators `&&` and `||` act on Boolean values and have short-circuit semantics.

Bit i can be selected using a mask with a single set bit, obtained in Rust as `1u64 << i`. Throughout these notes, whenever a shift uses a coordinate index, the index must be strictly less than the bit width of the type. The forms `^=`, `&=`, and `|=` update a variable by applying the corresponding bitwise operator.

Example 0.3 (Tracing shifts on an eight-bit word). Let $w = 0b10110101$ be the initial word in an illustrative eight-bit storage type. We compute $w \ll 2$ and $w \gg 2$ separately: the second diagram starts again from w , not from the result of the first. Two extra boxes show the discarded bits. We use eight bits only to display the entire string. The same rule applies to `u64`, with the boundary at bit 63.

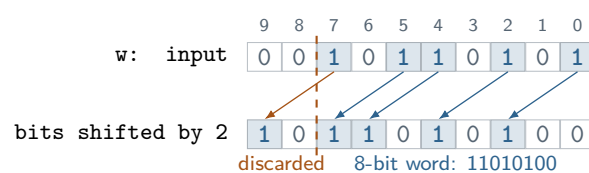


Figure 0.2: Left shift of an illustrative 8-bit word. The two bits to the left of the line are excluded from the result, while the two zeros on the right are inserted by the shift.

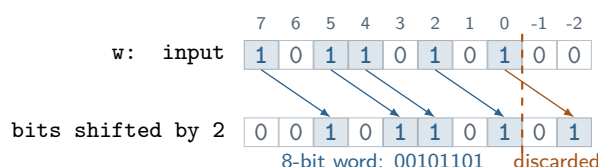


Figure 0.3: Right shift of the same word. Negative positions are drawn only to track discarded bits and are not positions in the word.

In Chapter 4, we will retain the discarded high bits separately because they still contribute to the result of polynomial reduction.

0.3 Implementation

The module `bits.rs` provides primitives for reading and updating coordinates in V_{64} and converting between words and bit arrays. For `get_bit`, `set_bit`, and `toggle_bit`, the precondition is $i < 64$. An out-of-range index is a caller error checked by `assert!`.

`get_bit(x: u64, i: usize) -> bool`. Returns coordinate i of the word x , counting from the least significant bit, whose index is 0. *Hint*: construct a mask that selects a single position, or move the desired coordinate to the least significant position before reading it.

`set_bit(x: u64, i: usize, value: bool) -> u64`. Returns a new word equal to x at every coordinate except the i th, which is set to `value`. *Hint*: use a mask with a single set bit and combine bitwise operations to clear or set that position without changing the others.

`toggle_bit(x: u64, i: usize) -> u64`. Returns a new word in which only coordinate i is flipped: 0 becomes 1 and 1 becomes 0. *Hint*: XOR with a mask containing a single set bit performs exactly this operation.

`to_bits(x: u64) -> [bool; 64]`. Returns all coordinates of the word in the order $(a_0, \dots, a_{63})$, from least to most significant. *Hint*: initialize an array to `false`, iterate over its indices, and reuse `get_bit`. The function `std::array::from_fn` is an alternative, not a requirement.

`from_bits(bits: [bool; 64]) -> u64`. Performs the inverse operation: `bits[i]` becomes coordinate i of the returned word. The type guarantees 64 Boolean coordinates. We pass it by value and, since it implements `Copy`, the original array remains available to the caller. We will use borrows or slices to read structures that we do not want to copy or consume, such as a `Vec` of coefficients. *Hint*: start with the zero word and insert only the true coordinates, using shifts and OR or reusing `set_bit`.

These signatures do not return recoverable errors. Conversions are total on the declared types. Do not introduce `Result` for conditions already excluded by the contract or type. Binary formatting and bit counting are outside this API: for debugging, use Rust's standard formatters directly and, when needed, `u64::count_ones()`.

For a mathematical width $m < 64$, use the first m coordinates and require all remaining coordinates to be zero whenever the value is declared canonical at width m . Concrete field APIs will encode this information in the type or field specification, so we do not add a `width` parameter to every call here.

0.4 Verification

The file `tests/bits.rs` contains eight tests with fixed inputs:

- `word_bits_round_trip` and `bits_word_round_trip` check the two conversions, and `thirteen_h as_expected_coordinates` establishes their order using the word 13.
- `get_bit_reads_the_same_coordinate_as_to_bits` compares direct access with conversion. The test `set_bit_changes_only_one_coordinate` checks a bit being set to true, and `toggle_bit_is_an_involution` checks that toggling a bit twice restores it.
- `bit_index_out_of_range_panics`, marked `#[should_panic]`, calls `get_bit(13, 64)`. The test `xor_matches_coordinate_addition` compares XOR with coordinate addition.

These tests document the cases actually checked but do not exhaust all inputs or all precondition violations.

Chapter 1

Binary Extensions: Degree, Basis, and a First Model

In `examples/gf4.rs`, we develop direct arithmetic in a quadratic extension. We construct a four-element field to see how an algebraic relation determines multiplication on coordinates, distinguishing the indeterminate X , its class α_4 , and the word encoding that class.

1.1 Algebra

For the general concepts, we follow [1, 2]. The identities needed for the example are proved here.

1.1.1 Extensions and vector-space structure

Let K and E be fields. A **field embedding**

$$\iota : K \hookrightarrow E$$

is a map that preserves addition, multiplication, and the multiplicative identity:

$$\iota(a + b) = \iota(a) + \iota(b), \quad \iota(ab) = \iota(a)\iota(b), \quad \iota(1_K) = 1_E.$$

When such an embedding exists, we call E an **extension** of K and write E/K . Here the symbol “/” **does not denote a quotient**: it only indicates that we regard E as an extension of the base field K .

Proposition 1 (A field embedding is injective). If $\iota : K \rightarrow E$ preserves addition, multiplication, and the multiplicative identity, then ι is injective.

Proof. Suppose $\iota(a) = 0$ for some $a \in K$. If $a \neq 0$, then a has an inverse $a^{-1} \in K$, and

$$1_E = \iota(1_K) = \iota(aa^{-1}) = \iota(a)\iota(a^{-1}) = 0,$$

a contradiction. Hence $a = 0$, so $\ker \iota = \{0\}$ and ι is injective. $\square$

Since ι is injective, we may identify K with the subfield $\iota(K) \subseteq E$ and write

$$K \subseteq E.$$

Elements of K therefore act as scalars on E : for $\lambda \in K$ and $x \in E$, define $\lambda \cdot x$ using multiplication in E .

For $\lambda, \mu \in K$ and $x, y \in E$, the field laws give

$$\begin{aligned} \lambda(x + y) &= \lambda x + \lambda y, & (\lambda + \mu)x &= \lambda x + \mu x, \\ (\lambda\mu)x &= \lambda(\mu x), & 1_K x &= 1_E x = x. \end{aligned}$$

Moreover, $(E, +)$ is an abelian group. Thus E , with its own addition and this scalar multiplication, is a **vector space over K** . We can now speak of the dimension of E over K .

1.1.2 Degree, bases, and coordinates

Definition 2 (Degree of an extension). The **degree of the extension** E/K is the dimension of E as a vector space over K :

$$[E : K] := \dim_K E.$$

If $[E : K] = n < \infty$, the extension is called finite.

Let

$$\mathcal{B} = (\beta_0, \dots, \beta_{n-1})$$

be a basis of E over K . By definition, the elements β_i span E and are linearly independent over K .

Proposition 3 (Existence and uniqueness of coordinates). Every $x \in E$ has a unique representation

$$x = \sum_{i=0}^{n-1} a_i \beta_i, \quad a_i \in K.$$

Proof. Since $\mathcal{B}$ spans E , for every $x \in E$ there are coefficients $a_i \in K$ such that $x = \sum_i a_i \beta_i$. Suppose there are two representations

$$x = \sum_i a_i \beta_i = \sum_i b_i \beta_i.$$

Subtracting gives

$$0 = \sum_i (a_i - b_i) \beta_i.$$

Linear independence of the basis implies $a_i - b_i = 0$ for every i , hence $a_i = b_i$ for every i . $\square$

Fixing the basis $\mathcal{B}$, define the coordinate map

$$\text{coord}_{\mathcal{B}} : E \longrightarrow K^n, \quad \text{coord}_{\mathcal{B}} \left(\sum_{i=0}^{n-1} a_i \beta_i \right) = (a_0, \dots, a_{n-1}).$$

Proposition 4 (The coordinate map is a vector-space isomorphism). The map $\text{coord}_{\mathcal{B}} : E \rightarrow K^n$ is K -linear and bijective.

Proof. If $x = \sum_i a_i \beta_i$ and $y = \sum_i b_i \beta_i$, then

$$\text{coord}_{\mathcal{B}}(x + y) = (a_0 + b_0, \dots, a_{n-1} + b_{n-1}) = \text{coord}_{\mathcal{B}}(x) + \text{coord}_{\mathcal{B}}(y).$$

For $\lambda \in K$,

$$\text{coord}_{\mathcal{B}}(\lambda x) = (\lambda a_0, \dots, \lambda a_{n-1}) = \lambda \text{coord}_{\mathcal{B}}(x),$$

so the map is linear. If two elements have the same coordinates, the uniqueness just proved makes them equal, so the map is injective. Finally, every tuple $(a_0, \dots, a_{n-1}) \in K^n$ is the image of $\sum_i a_i \beta_i$, so the map is surjective. $\square$

Therefore

$$E \simeq K^n$$

as vector spaces over K . This is not, in general, a field or ring isomorphism. Coordinates immediately determine addition, but do not by themselves determine multiplication in E .

Corollary 5 (Cardinality of a finite extension). If $|K| = q$ and $[E : K] = n < \infty$, then

$$|E| = q^n.$$

Proof. The coordinate map is a bijection $E \leftrightarrow K^n$. Each of the n coordinates has q possible values, so $|K^n| = q^n$ and therefore $|E| = q^n$. $\square$

1.1.3 Characteristic of an extension

The characteristic of a field is the least positive integer n such that $n \cdot 1 = 0$, if one exists, and 0 otherwise.

Proposition 6 (Extensions preserve characteristic). If $K \hookrightarrow E$ is a field extension, then

$$\boxed{\text{char } E = \text{char } K}.$$

Proof. For every integer $n \geq 1$,

$$\iota(n \cdot 1_K) = n \cdot \iota(1_K) = n \cdot 1_E.$$

If $n \cdot 1_K = 0$, applying ι gives $n \cdot 1_E = 0$. Conversely, if $n \cdot 1_E = 0$, then $\iota(n \cdot 1_K) = 0$. Since ι is injective, $n \cdot 1_K = 0$. Thus the positive integers that annihilate the multiplicative identity are the same in both fields. $\square$

In particular, every extension of $\mathbb{F}_2$ has characteristic 2: $x + x = 0$ and $-x = x$.

1.1.4 Constructing the quadratic model

We want to construct a degree-2 extension of

$$\mathbb{F}_2 = \{0, 1\}.$$

Since $|\mathbb{F}_2| = 2$, such an extension must have $2^2 = 4$ elements. We therefore seek a field whose elements are

$$0, \quad 1, \quad \alpha_4, \quad 1 + \alpha_4,$$

and must determine its multiplication, that is, the value of α_4^2 .

Consider the polynomial

$$p_2(X) = X^2 + X + 1 \in \mathbb{F}_2[X].$$

Here X is a formal indeterminate. For this section, it is enough to know that an element of $\mathbb{F}_2[X]$ is a polynomial $a_0 + a_1X + \cdots + a_dX^d$ with coefficients in $\mathbb{F}_2$. The general theory of polynomials is developed in the following sections.

Proposition 7 (Irreducibility of $X^2 + X + 1$). The polynomial $p_2(X) = X^2 + X + 1$ is irreducible over $\mathbb{F}_2$.

Proof. The only elements of $\mathbb{F}_2$ are 0 and 1, and

$$p_2(0) = 1, \quad p_2(1) = 1 + 1 + 1 = 1.$$

Thus $p_2(X)$ has no roots in $\mathbb{F}_2$. A reducible quadratic polynomial over a field must factor as a product of two degree-one polynomials, and must therefore have a root in that field. Hence $p_2(X)$ is irreducible. $\square$

We choose the model $\mathbb{F}_4 := \mathbb{F}_2[X]/(p_2)$. We call $A(X)$ and $B(X)$ equivalent if $p_2(X)$ divides $A(X) - B(X)$, and write $[A]$ for the class of A . Define

$$[A] + [B] = [A + B], \quad [A][B] = [AB].$$

These operations are well-defined: changing representatives by multiples of p_2 changes their sum and product by multiples of p_2 . For the product, this follows from the identity

$$A'B' - AB = (A' - A)B' + A(B' - B).$$

The ring laws pass to the classes. Here we use only this concrete case. Chapter 4 develops the construction over an arbitrary field.

Set $\alpha_4 = [X]$. Since p_2 is a multiple of itself, $[p_2] = [0]$, and hence

$$0 = [X^2 + X + 1] = \alpha_4^2 + \alpha_4 + 1, \quad \boxed{\alpha_4^2 = \alpha_4 + 1}.$$

Here X remains a formal symbol, whereas α_4 is an element of the quotient. This relation lets us compute without introducing a general division algorithm.

1.1.5 Representatives and inverses in the quadratic model

The relation $\alpha_4^2 = \alpha_4 + 1$ eliminates all powers α_4^k with $k \geq 2$.

Example 1.1 (Reducing the first powers of α_4).

$$\alpha_4^2 = \alpha_4 + 1, \quad \alpha_4^3 = \alpha_4(\alpha_4 + 1) = \alpha_4^2 + \alpha_4 = 1, \quad \alpha_4^4 = \alpha_4.$$

More generally, $\alpha_4^0 = 1$ and $\alpha_4^1 = \alpha_4$. For $k \geq 2$,

$$\alpha_4^k = \alpha_4^{k-2} \alpha_4^2 = \alpha_4^{k-1} + \alpha_4^{k-2}.$$

By induction, every power, and hence every polynomial in α_4 , has the form

$$\boxed{a + b\alpha_4, \quad a, b \in \mathbb{F}_2}.$$

The representation is unique. If

$$a + b\alpha_4 = c + d\alpha_4,$$

then the polynomial

$$(a - c) + (b - d)\alpha_4$$

represents the zero class and is therefore a multiple of $p_2(X)$. Its degree is at most one, whereas a nonzero multiple of a monic quadratic has degree at least two. It must therefore be the zero polynomial, giving $a = c$, $b = d$.

This yields exactly the four classes

$$\boxed{0, \quad 1, \quad \alpha_4, \quad 1 + \alpha_4}.$$

The family $(1, \alpha_4)$ spans the quotient and is linearly independent by uniqueness of the representation. Therefore

$$\boxed{(1, \alpha_4) \text{ is a basis of } \mathbb{F}_4/\mathbb{F}_2}, \quad [\mathbb{F}_4 : \mathbb{F}_2] = 2, \quad |\mathbb{F}_4| = 4.$$

We have constructed a ring with four elements. For this ring to be a field, every nonzero element must have an inverse. For 1, we have $1^{-1} = 1$, while

$$\alpha_4(\alpha_4 + 1) = \alpha_4^2 + \alpha_4 = (\alpha_4 + 1) + \alpha_4 = 1.$$

Hence

$$\alpha_4^{-1} = \alpha_4 + 1, \quad (\alpha_4 + 1)^{-1} = \alpha_4.$$

All nonzero elements are invertible. Thus

$$\boxed{\mathbb{F}_4 := \mathbb{F}_2[X]/(X^2 + X + 1)}$$

is a field, giving a concrete extension $\mathbb{F}_2 \subseteq \mathbb{F}_4$ of degree 2. Chapter 4 generalizes this construction by proving

$$K[X]/(p(X)) \text{ is a field} \iff p(X) \text{ is irreducible.}$$

1.1.6 Addition and multiplication in the quadratic model

Let $a = a_0 + a_1\alpha_4$ and $b = b_0 + b_1\alpha_4$, with $a_i, b_i \in \mathbb{F}_2$. Addition combines coordinates with the same index:

$$a + b = (a_0 + b_0) + (a_1 + b_1)\alpha_4.$$

For multiplication, first expand and then use $\alpha_4^2 = \alpha_4 + 1$:

$$\begin{aligned} ab &= a_0b_0 + (a_0b_1 + a_1b_0)\alpha_4 + a_1b_1\alpha_4^2 \\ &= (a_0b_0 + a_1b_1) + (a_0b_1 + a_1b_0 + a_1b_1)\alpha_4. \end{aligned}$$

If $ab = c_0 + c_1\alpha_4$, the required coordinates are

$$\boxed{c_0 = a_0b_0 + a_1b_1, \quad c_1 = a_0b_1 + a_1b_0 + a_1b_1.}$$

Each product a_ib_j is a product of coefficients in $\mathbb{F}_2$, so it is implemented with AND, while each coefficient sum is implemented with XOR. Multiplying the elements a, b requires these cross terms, rather than coordinatewise AND on their words.

1.2 Representation

Fix the ordered basis

$$\mathcal{B} = (1, \alpha_4).$$

For $a = a_0 + a_1\alpha_4$, we have

$$\text{coord}_{\mathcal{B}}(a) = (a_0, a_1), \quad \text{enc}_{\mathcal{B}}(a_0 + a_1\alpha_4) = \tilde{a}_0 + 2\tilde{a}_1.$$

Here $\tilde{a}_i \in \{0, 1\} \subset \mathbb{Z}$ is the integer digit associated with the coefficient a_i . This gives

Element	Coordinates (a_0, a_1)	Word
0	(0, 0)	0b00
1	(1, 0)	0b01
α_4	(0, 1)	0b10
$1 + \alpha_4$	(1, 1)	0b11

XOR adds the two coordinates separately in $\mathbb{F}_2$, so

$$\boxed{\text{enc}_{\mathcal{B}}(a + b) = \text{enc}_{\mathcal{B}}(a) \oplus \text{enc}_{\mathcal{B}}(b)}.$$

Example 1.2 (Coordinatewise AND and field multiplication).

$$\text{coord}_{\mathcal{B}}(\alpha_4) = (0, 1), \quad (0, 1) \wedge (0, 1) = (0, 1),$$

where $\wedge$ acts coordinatewise. In the field, however,

$$\alpha_4^2 = \alpha_4 + 1, \quad \text{coord}_{\mathcal{B}}(\alpha_4^2) = (1, 1).$$

Therefore

$$\boxed{\text{coordinatewise product } \wedge \neq \text{field product}}.$$

In the Rust representation, the first calculation corresponds to $0b10 \ \& \ 0b10 = 0b10$, while the field square is encoded by $0b11$.

The encoding is the composition of the coordinate map with the packing operation from Chapter 0. This example uses only two bits, while the degree-128 model will use 128.

1.3 Implementation

Create `examples/gf4.rs`. This is an executable Cargo example, not a public library module: use a small `main` to try its operations, and import the bit primitives from `binary_afft::bits`. Do not declare a `gf4` module in `src/lib.rs` [5]. The model is already fixed in this chapter: function inputs are the two binary coordinates in the basis $(1, \alpha_4)$, packed into the two least significant bits of a `u8`. The allowed masks are therefore 0, 1, 2, 3. Other values violate the contract and are rejected by `assert!`.

`gf4_add(left: u8, right: u8) -> u8`. Returns the encoding of $a + b$ when `left` and `right` represent $a, b \in \mathbb{F}_4$. Since the basis is fixed and the coefficients lie in $\mathbb{F}_2$, addition is coordinatewise. *Hint*: use the bitwise operator that implements addition in $\mathbb{F}_2$ directly. Do not use integer addition with carries.

`gf4_mul(left: u8, right: u8) -> u8`. Returns the encoding of the field product. Interpret the two bits of each operand as the coefficients of $a_0 + a_1\alpha_4$ and $b_0 + b_1\alpha_4$, expand the symbolic product, substitute $\alpha_4^2 = \alpha_4 + 1$, and pack the two resulting coordinates. *Hint*: reuse the bit primitives from Chapter 0 to read the coordinates. Avoid a hard-coded multiplication table: the exercise is meant to translate the algebraic relation into a computation.

These functions are deliberately concrete. Do not introduce a generic extension type here or call the later degree-128 reduction. The direct implementation of $\mathbb{F}_4$ remains a self-contained teaching example. It is not automatically a reference implementation for $\mathbb{F}_{2^{128}}$: identical masks in different bases do not define an embedding.

1.4 Verification

The six tests in `examples/gf4.rs` form a local `tests` module enabled by `#[cfg(test)]`. The statement `use super::*`; makes the parent module's functions accessible [6, 7]. There are only four values here, so the addition and multiplicative-closure tests examine all 16 pairs.

The test `gf4_addition_is_coordinatewise_xor` compares addition with XOR. To reconstruct multiplication by hand, use $\alpha_4^2 = \alpha_4 + 1$ to obtain the following table:

<code>gf4_mul</code>	0	1	2	3
0	0	0	0	0
1	0	1	2	3
2	0	2	3	1
3	0	3	1	2

The labels 0, 1, 2, 3 are the words encoding $0, 1, \alpha_4, 1 + \alpha_4$, not factors in an integer multiplication. This table is a teaching reference and is not hard-coded in the current test. The repository checks the generator relation with `gf4_generator_satisfies_defining_relation`, closure with `gf4_multiplication_stays_in_the_field`, and the field laws with `gf4_field_laws_hold`, which also examines all triples. The tests `gf4_add_rejects_noncanonical_masks` and `gf4_mul_rejects_noncanonical_masks` check that noncanonical masks cause a panic.

Run `cargo test --example gf4`. The command `cargo test --all-targets` also includes example tests. The command `cargo test` alone does not run example tests [5].

Chapter 2

Formal Polynomials and Binary Encoding

In the `polynomial.rs` module, we develop the computation of a polynomial's degree.

2.1 Algebra

Let K be a field. A **formal polynomial** over K is defined as a sequence of coefficients

$$A = (a_0, a_1, a_2, \dots), \quad a_i \in K,$$

with **finite support**, rather than as a function $K \rightarrow K$. Setting

$$\text{supp}(A) := \{i \geq 0 : a_i \neq 0\},$$

the condition means that $\text{supp}(A)$ contains only finitely many indices. The usual notation

$$A(X) = a_0 + a_1X + a_2X^2 + \dots = \sum_{i \geq 0} a_i X^i$$

is a convenient way to represent this sequence. The symbol X is a **formal indeterminate**: at this stage, it represents neither an element of K nor a chosen root in an extension.

The distinction between a polynomial and a function matters, particularly over finite fields.

Example 2.1 (Distinct polynomials inducing the same function). In $\mathbb{F}_2[X]$, the nonzero polynomial

$$X^2 + X$$

vanishes at both 0 and 1, so it induces the same function $\mathbb{F}_2 \rightarrow \mathbb{F}_2$ as the zero polynomial. The two polynomials remain distinct because their coefficients differ. Evaluating a polynomial is therefore different from identifying it by its coefficients.

Since a polynomial is determined by its coefficient sequence,

$$A = B \iff a_i = b_i \quad \text{for every } i \geq 0.$$

If $B(X) = \sum_{i \geq 0} b_i X^i$, addition is defined coordinatewise:

$$A(X) + B(X) = \sum_{i \geq 0} (a_i + b_i) X^i,$$

hence

$$(A + B)_k = a_k + b_k.$$

The support of the sum is contained in the union of the two supports, so it remains finite.

Multiplication, in turn, is defined by the *Cauchy product*, that is, coefficient convolution. Formally, using distributivity and the rule $X^i X^j = X^{i+j}$,

$$\begin{aligned} A(X)B(X) &= \left(\sum_i a_i X^i \right) \left(\sum_j b_j X^j \right) \\ &= \sum_{i,j} a_i b_j X^{i+j} \\ &= \sum_{k \geq 0} \left(\sum_{i+j=k} a_i b_j \right) X^k. \end{aligned}$$

The coefficient of X^k is therefore

$$(AB)_k = \sum_{i+j=k} a_i b_j.$$

For a fixed k , only the pairs

$$(0, k), (1, k-1), \dots, (k, 0),$$

contribute. Since the supports of A and B are finite, the product also has finite support. With these operations, $K[X]$ is a commutative ring with identity.

This definition is already close to a software representation. A list

$$[a_0, a_1, \dots, a_d]$$

can store the coefficients of A : index i corresponds to the coefficient of X^i . Addition combines matching positions, while multiplication combines a pair of positions (i, j) and accumulates the contribution at position $i + j$. For binary coefficients, this becomes a convolution implemented with XOR and bit products. Chapter 3 translates precisely this definition into the carry-less algorithm.

For $A \neq 0$, define

$$\deg A := \max\{i : a_i \neq 0\}.$$

The zero polynomial has no degree. A nonzero polynomial is monic if its coefficient at index $\deg A$ is 1.

Proposition 8 (Degree of a product). If $A, B \in K[X]$ are nonzero, then

$$\deg(AB) = \deg A + \deg B.$$

Proof. Let $d = \deg A$ and $e = \deg B$. The coefficient of X^{d+e} in AB is $a_d b_e$, which is nonzero because K is a field. No term of higher degree can occur. Thus $d + e$ is the largest index with a nonzero coefficient. $\square$

To include the zero polynomial without assigning it a degree, we use the notation

$$K[X]_{<n} := \{0\} \cup \{A \in K[X] \setminus \{0\} : \deg A < n\} \quad (n \geq 0).$$

In particular, $K[X]_{<0} = \{0\}$. Whenever the context allows it, a bound on polynomial degree will also include the zero polynomial. Proofs involving the degree treat zero separately.

2.2 Representation

We now specialize to $K = \mathbb{F}_2$. For polynomials of degree less than 128, define

$$\text{enc}_{128} \left(\sum_{i=0}^{127} a_i X^i \right) := \sum_{i=0}^{127} \tilde{a}_i 2^i \in \{0, \dots, 2^{128} - 1\},$$

where $\tilde{a}_i \in \{0, 1\} \subset \mathbb{Z}$ represents the coefficient a_i . The map is a bijection onto the values of type `u128`, which canonically encode the polynomials in this space.

If a word w has bits $w_0, \dots, w_{127} \in \mathbb{F}_2$, we simply say that w encodes the polynomial

$$w_0 + w_1 X + \dots + w_{127} X^{127}.$$

This is a convention for reading coefficients, not a new API. The same convention applies to a `u64`, with zero coefficients from index 64 onward.

The encoding directly extends the bit convention from Chapter 0:

$$\text{enc}_{128}(A + B) = \text{enc}_{128}(A) \oplus \text{enc}_{128}(B).$$

Example 2.2 (Encoding two binary polynomials). Let $A(X) = X^3 + X + 1$ and $B(X) = X^4 + X + 1$ be the two polynomials to encode, with words **a** and **b**:

$$0b1011 \longleftrightarrow X^3 + X + 1, \quad 0b10011 \longleftrightarrow X^4 + X + 1.$$

a: word for $A(X)$

7	6	5	4	3	2	1	0
0	0	0	0	1	0	1	1

b: word for $B(X)$

7	6	5	4	3	2	1	0
0	0	0	1	0	0	1	1

Figure 2.1: Rows **a** and **b** are separate encodings. The index above each box is the monomial degree. Omitted coefficients, from 8 to 127, are zero.

Integer multiplication `*` does not represent multiplication in $\mathbb{F}_2[X]$. Moreover, the product of two polynomials representable in `u128` can have degree greater than 127. Chapter 3 therefore introduces two total operations: $64 \times 64 \rightarrow 128$ bits and $128 \times 128 \rightarrow (128 + 128)$ bits, both without coefficient loss.

2.3 Implementation

The only new primitive in this chapter, to be written in `src/polynomial.rs`, is

```
poly_degree(p: u128) -> Option<usize>.
```

The function returns `None` exactly for the zero polynomial. For a nonzero polynomial, it returns the index of the highest-degree nonzero coefficient. The number of set bits is irrelevant: a polynomial can have low Hamming weight and high degree.

Hint: find the set bit with the largest index. You can first write an explicit version that scans the positions, verify it, and then compare it with the standard `u128` primitives for counting leading zeros. Do not convert through a binary string: the internal representation is already numeric.

Do not implement multiplication, evaluation, or reduction in this chapter. Each is introduced only after its algebraic operation has been defined.

2.4 Verification

The file `tests/polynomial.rs` contains four tests: `zero_polynomial_has_no_degree` checks `None` for zero, and `non_zero_constant_polynomial_has_degree_0` checks the constant 1. The test `degree_tracks_the_highest_power` uses `13` and `1u128 << 127`, while `degree_is_not_hamming_weight` checks that `0b10000001` has degree 7.

The tests are in `tests/polynomial.rs`.

Chapter 3

Carry-Less Polynomial Multiplication

In the `field/portable.rs` module, we construct `clmul` and `clmul128`, two distinct operations in sequence:

$$\text{clmul} : 64 \times 64 \longrightarrow 128 \text{ bits},$$

which directly computes the product of two binary polynomials of degree less than 64, followed by

$$\text{clmul128} : 128 \times 128 \longrightarrow 256 \text{ bits},$$

which reuses `clmul` as its building block.

The progression is therefore

$$\boxed{\text{convolution} \longrightarrow \text{clmul} \longrightarrow \text{two halves} \longrightarrow \text{Karatsuba} \longrightarrow (\text{lo}, \text{hi}).}$$

Neither function yet performs reduction modulo an irreducible polynomial.

3.1 Algebra

3.1.1 Carry-less multiplication and `clmul`

Let

$$a(X) = \sum_{i=0}^{63} a_i X^i, \quad b(X) = \sum_{j=0}^{63} b_j X^j, \quad a_i, b_j \in \mathbb{F}_2.$$

Write

$$c(X) = a(X)b(X) = \sum_{k=0}^{126} c_k X^k.$$

Algebraically, the coefficient of X^k is

$$\boxed{c_k = \sum_{i+j=k} a_i b_j.}$$

At the bit level, addition in $\mathbb{F}_2$ becomes XOR and multiplication in $\mathbb{F}_2$ becomes AND:

$$\boxed{c_k = \bigoplus_{i+j=k} (a_i \wedge b_j).}$$

There are no carries: two contributions to the same power cancel because $1 + 1 = 0$ in $\mathbb{F}_2$.

Now group the terms by the coefficients of the second factor:

$$\boxed{a(X)b(X) = \sum_{j: b_j=1} a(X)X^j.}$$

For each coefficient $b_j = 1$, we must add a copy of $a(X)$ shifted by j positions. On a word, this is a left shift.

Since $\deg a < 64$ and $j \leq 63$,

$$\deg(a(X)X^j) \leq 126,$$

so every shifted copy and the full result fit in a `u128`. This is precisely the contract of

$$\text{clmul}(\text{a: u64}, \text{b: u64}) \rightarrow \text{u128}.$$

While scanning the coefficients $b_0, b_1, \dots, b_{63}$, after processing indices $0, \dots, k-1$ the accumulator must encode

$$a(X) \sum_{j < k} b_j X^j.$$

If $b_k = 1$, add $a(X)X^k$. If $b_k = 0$, leave the accumulator unchanged. At the end, it encodes $a(X)b(X)$.

Example 3.1 (Carry-less multiplication and shift selection). Set

$$a(X) = X^3 + X + 1, \quad b(X) = X^2 + X,$$

with words $\mathbf{a} = 0\mathbf{b}1011$ and $\mathbf{b} = 0\mathbf{b}0110$. Only the coefficients of X and X^2 are nonzero in the second factor, so

$$c(X) = a(X)b(X) = a(X)X + a(X)X^2.$$

The code therefore computes only

$$\mathbf{t1} = \mathbf{a} \ll 1, \quad \mathbf{t2} = \mathbf{a} \ll 2, \quad \mathbf{c} = \mathbf{t1} \wedge \mathbf{t2}.$$

	7	6	5	4	3	2	1	0	
$\mathbf{a} \ll 1$	0	0	0	1	0	1	1	0	$a(X)X$
$\mathbf{a} \ll 2$	0	0	1	0	1	1	0	0	$a(X)X^2$
XOR	0	0	1	1	1	0	1	0	$c(X)$

Figure 3.1: Only the two rows above the line are XOR operands. At degree 2, we obtain $1 \oplus 1 = 0$, with no carry.

3.1.2 From 128×128 to 64×64 products

Now consider two polynomials

$$a, b \in \mathbb{F}_2[X]_{<128}.$$

Split each into its low and high halves:

$$a = a_0 + a_1 X^{64}, \quad b = b_0 + b_1 X^{64},$$

where

$$a_0, a_1, b_0, b_1 \in \mathbb{F}_2[X]_{<64}.$$

By distributivity,

$$\begin{aligned} ab &= (a_0 + a_1 X^{64})(b_0 + b_1 X^{64}) \\ &= a_0 b_0 + (a_0 b_1 + a_1 b_0) X^{64} + a_1 b_1 X^{128}. \end{aligned}$$

Define

$$p_0 = a_0 b_0, \quad p_1 = a_0 b_1 + a_1 b_0, \quad p_2 = a_1 b_1.$$

Then

$$ab = p_0 + p_1 X^{64} + p_2 X^{128}.$$

The direct formula would require four 64×64 products: one for p_0 , two for the summands of p_1 , and one for p_2 .

3.1.3 Karatsuba: obtaining p_1 with one product

The identity

$$(a_0 + a_1)(b_0 + b_1) = a_0b_0 + a_0b_1 + a_1b_0 + a_1b_1 = p_0 + p_1 + p_2$$

implies, in characteristic two,

$$p_1 = (a_0 + a_1)(b_0 + b_1) + p_0 + p_2.$$

Thus three calls to `clmul` suffice:

$$\begin{aligned} p_0 &= a_0b_0, \\ p_2 &= a_1b_1, \\ p_1 &= (a_0 + a_1)(b_0 + b_1) + p_0 + p_2. \end{aligned}$$

The sums $a_0 + a_1$ and $b_0 + b_1$ are XORs of the corresponding words. Each p_i has degree less than 128 and therefore fits in a `u128`.

3.2 Representation

3.2.1 Input halves

The word `a`: `u128` encodes

$$a = a_0 + a_1X^{64}.$$

In code,

$$\text{a0} = \text{a as u64}$$

retains the low 64 bits, while

$$\text{a1} = (\text{a} \gg 64) \text{ as u64}$$

moves the high 64 bits to positions $0, \dots, 63$. Obtain `b0` and `b1` in the same way. These four words encode exactly the four polynomial blocks a_0, a_1, b_0, b_1 in the preceding decomposition.

3.2.2 Why the middle term crosses the boundary between `lo` and `hi`

The full product can have degree 254, so it does not fit in a single `u128`. Write

$$ab = (ab)_0 + (ab)_1X^{128},$$

with

$$(ab)_0, (ab)_1 \in \mathbb{F}_2[X]_{<128}.$$

The word `lo` encodes $(ab)_0$, and `hi` encodes $(ab)_1$.

Start from the formula

$$ab = p_0 + p_1X^{64} + p_2X^{128}.$$

The terms p_0 and p_2X^{128} are already aligned with the low and high halves, respectively. The only term crossing the boundary at degree 128 is

$$p_1X^{64}.$$

To split it, write

$$p_1 = p_{1,0} + p_{1,1}X^{64}, \quad p_{1,0}, p_{1,1} \in \mathbb{F}_2[X]_{<64}.$$

Multiplying by X^{64} gives

$$p_1X^{64} = p_{1,0}X^{64} + p_{1,1}X^{128}.$$

The first summand still lies in degrees $0, \dots, 127$ and therefore contributes to $(ab)_0$. The second has a factor X^{128} and contributes to $(ab)_1$.

Thus

$$(ab)_0 = p_0 + p_{1,0}X^{64}, \quad (ab)_1 = p_2 + p_{1,1}.$$

The same split is visible directly in the word `p1`. View it as two 64-bit blocks:

bits 127...64	bits 63...0
$p_{1,1}$	$p_{1,0}$

Left and right shifts extract the two blocks in different ways:

	bits 127...64	bits 63...0
<code>p1</code>	$p_{1,1}$	$p_{1,0}$
<code>p1 << 64</code>	$p_{1,0}$	0
<code>p1 >> 64</code>	0	$p_{1,1}$

Hence:

- `p1 << 64` takes the low half of p_1 and places it in the high half of `lo`. The high half of p_1 leaves the word because it does not belong in `lo`.
- `p1 >> 64` takes the high half of p_1 and reindexes it into the low bits, ready to be added to p_2 in `hi`.

The final recombination is therefore

$$\begin{aligned} \text{lo} &= \text{p0} \oplus (\text{p1} \ll 64), \\ \text{hi} &= \text{p2} \oplus (\text{p1} \gg 64). \end{aligned}$$

Both shifts act on the original word `p1` and are not applied sequentially.

3.3 Implementation

Multiplication is still an operation in $\mathbb{F}_2[X]$, but in the project it will serve as a primitive of the portable backend for $\mathbb{F}_{2^{128}}$. This is why the file is already `src/field/portable.rs`.

Create the directory `src/field/`. At this stage, the two module declaration files are complete as follows:

src/lib.rs

```
pub mod bits;
pub mod field;
pub mod polynomial;
```

src/field/mod.rs

```
pub mod portable;
```

The declaration `pub mod field;` loads `field/mod.rs`. Within it, `pub mod portable;` loads `field/portable.rs`. Do not declare `gf128` yet: the type is introduced in Chapter 5 [7].

3.3.1 Implement `clmul` first

In `src/field/portable.rs`, first implement

```
pub fn clmul(a: u64, b: u64) -> u128.
```

The algorithm directly expresses the formula

$$a(X)b(X) = \sum_{j:b_j=1} a(X)X^j.$$

Use a `u128` accumulator initialized to zero. Widen `a` to `u128` before performing any shift. For each $j = 0, \dots, 63$:

1. Read bit j of `b`, using the existing `get_bit` primitive for `u64` or directly evaluating `((b >> j) & 1) != 0`.
2. If the bit is 1, XOR the accumulator with `(a as u128) << j`.
3. If the bit is 0, leave the accumulator unchanged.

Return the accumulator. Do not use integer multiplication `*`: it would introduce carries and compute a different operation.

3.3.2 Then implement `clmul128`

Only after verifying `clmul`, implement

```
pub fn clmul128(a: u128, b: u128) -> (u128, u128).
```

The function has four steps:

1. Extract `a0`, `a1`, `b0`, `b1`.
2. Compute `p0 = clmul(a0, b0)` and `p2 = clmul(a1, b1)`.
3. Apply Karatsuba:

$$p1 = \text{clmul}(a0 \oplus a1, b0 \oplus b1) \oplus p0 \oplus p2.$$

4. Recombine:

$$lo = p0 \oplus (p1 \ll 64), \quad hi = p2 \oplus (p1 \gg 64).$$

Return `(lo, hi)` in this order.

3.4 Verification

In `tests/field_portable.rs`, multiplication needs a single test, `clmuls_match_reference`, which checks both `clmul` and `clmul128` in the same loop. Import both from `binary_ffft::field::portable`, not from `polynomial`.

Use the coefficient-based reference implementation in `tests/common/slow_clmul.rs`, with signature `slow_clmul(a: u128, b: u128) -> (u128, u128)`. Two loops over $i, j < 128$ accumulate the AND contribution of bits i and j into an array of 256 Booleans, using XOR at position $i + j$. Then reconstruct the two words from coefficients 0–127 and 128–255. The reference implementation calls neither `clmul` nor `clmul128`. It reads bits with shifts and the mask `& 1`, without adding public APIs.

Over 1000 iterations, first generate two `u64` values and compare `clmul(a, b)` with the low half returned by `slow_clmul`, and then generate two `u128` values and compare the entire pair returned by `clmul128` with the same reference implementation. This independent comparison directly checks the result defined by binary convolution.

The tests are in `tests/field_portable.rs`.

The construction of the wide product is now complete: `clmul128` uses `clmul` three times and returns `(lo, hi)`. The next chapter reduces this pair to obtain the field product.

Part II

Arithmetic in $\mathbb{F}_{2^{128}}$

Chapter 4

Field Construction and Specialized Reduction

In the `field/portable.rs` module, we now develop the reduction function `reduce_p128`.

The preceding chapter solved polynomial multiplication: `clmul128` returns every coefficient of $C(X) = A(X)B(X)$ in the pair `(lo, hi)`. We must now obtain the single word representing the field product. Simply discarding `hi` is not enough: the high coefficients also contribute through the relation imposed by the modulus.

The theory determines *what to compute*: the remainder modulo an irreducible polynomial of degree 128. The particular form of the modulus determines *how to compute it*: replace the entire high part of the product with a few contributions of lower degree. This step is a *fold*. We derive it first for polynomials and then for words, without introducing a generic division library.

4.1 Algebra

4.1.1 Euclidean division and the meaning of the remainder

Euclidean division separates a polynomial into a multiple of the divisor and a part of lower degree. The latter is the representative we store.

Theorem 9 (Euclidean division for polynomials over a field). Let $C, D \in K[X]$, where K is a field and $D \neq 0$. There is a unique pair $Q, R \in K[X]$ such that

$$C = QD + R, \quad R = 0 \text{ or } \deg R < \deg D.$$

The polynomial R is called the remainder of C modulo D and is denoted by $C \bmod D$.

Proof. Existence. Initialize $Q = 0$ and $R = C$, preserving the identity $C = QD + R$. If $R \neq 0$ and $\deg R \geq \deg D$, let c, b be the leading coefficients of R, D . Set

$$s = \deg R - \deg D, \quad S = \frac{c}{b}X^s.$$

The product SD has the same leading term as R . Replacing Q by $Q + S$ and R by $R - SD$ cancels that term and preserves the identity:

$$(Q + S)D + (R - SD) = QD + R = C.$$

Each successive remainder is zero or has strictly lower degree than the preceding one. The process therefore terminates with an admissible remainder.

Uniqueness. If $C = QD + R = Q'D + R'$, then $(Q - Q')D = R' - R$. If $Q \neq Q'$, the left-hand side is nonzero and has degree at least $\deg D$, whereas the right-hand side is zero or has degree less than $\deg D$. This contradiction forces $Q = Q'$, and hence $R = R'$. $\square$

The proof suggests long division: at each step, align the leading term of the divisor with that of the current remainder and cancel it. The shift exponent is $s = \deg R - \deg D$, *not* $\deg R$.

Example 4.1 (Degree alignment in long division). To cancel a term of degree 133 using a divisor of degree 128, take $s = 133 - 128 = 5$ and multiply the divisor by X^5 .

In $\mathbb{F}_2[X]$, every nonzero leading coefficient is 1, and subtraction equals addition. The remainder update becomes $R \leftarrow R + DX^s$: addition of binary coefficients is XOR. This procedure will serve as an independent reference in the tests. For field multiplication, we instead use an algorithm that exploits the fixed divisor.

4.1.2 Polynomial classes and the field construction

Fix a monic polynomial $p \in K[X]$ of degree $m \geq 1$. We regard two polynomials as equivalent if their difference is a multiple of p . Write

$$A \equiv B \pmod{p} \iff p \mid (A - B), \quad [A] = \{A + pJ : J \in K[X]\}.$$

This is an equivalence relation because 0, additive inverses of multiples of p , and sums of such multiples are again multiples of p . The set of classes is $K[X]/(p)$, where (p) denotes the ideal of multiples of p .

Define $[A] + [B] = [A + B]$ and $[A][B] = [AB]$. These operations are independent of the representatives: if $A' - A$ and $B' - B$ are multiples of p , then so are $(A' + B') - (A + B)$ and

$$A'B' - AB = (A' - A)B' + A(B' - B).$$

The ring laws therefore pass to the classes. Zero and the multiplicative identity are distinct because the nonconstant polynomial p does not divide 1.

Euclidean division of A gives $A = Qp + R$ with $R \in K[X]_{<m}$, hence $[A] = [R]$. Each class has exactly one representative of degree less than m , including the zero polynomial. Reduction changes the polynomial representing the class, not the class itself. For multiplication, this means

$$[A][B] = [AB] = [AB \bmod p]. \quad (4.1)$$

We can therefore multiply representatives in the ring and then compute the remainder: this is exactly the composition we will implement.

An arbitrary choice of p , however, does not suffice to obtain a field. A polynomial of positive degree is *irreducible over K* if every factorization in $K[X]$ has a constant factor. The following result shows why this guarantees that nonzero classes are invertible. We choose the greatest common divisor to be monic: it divides both polynomials and is divisible by each of their common divisors.

Lemma 10 (Bézout's identity for polynomials). If $A, D \in K[X]$ are not both zero, there exist $U, V \in K[X]$ such that $UA + VD = \gcd(A, D)$. In particular, if A, D are coprime, there exist U, V such that $UA + VD = 1$.

Proof. Among the nonzero combinations $UA + VD$, choose one of least degree, normalize it to be monic, and call it G . Dividing A by G gives $A = QG + R$. If $G = UA + VD$, then

$$R = A - QG = (1 - QU)A - QVD.$$

The remainder is again a combination of A, D . If nonzero, it would have degree less than G , contradicting the choice of G . Thus $G \mid A$ and, by the same argument, $G \mid D$. Every common divisor also divides the combination G , which establishes the required property of the monic greatest common divisor. $\square$

Theorem 11 (When the quotient is a field). The quotient $K[X]/(p)$ is a field if and only if p is irreducible. In this case, setting $\alpha = [X]$, the constant classes identify K with a subfield, and

$$p(\alpha) = 0, \quad (1, \alpha, \dots, \alpha^{m-1}) \text{ is a basis over } K.$$

In particular, $[K[X]/(p) : K] = m$.

Proof. If p is irreducible and $[A] \neq [0]$, then $p \nmid A$. The monic greatest common divisor of A, p is therefore 1. Bézout's identity gives $UA + Vp = 1$. Passing to classes yields $[U][A] = [1]$. Thus every nonzero class has an inverse.

Conversely, if $p = UV$ with $0 < \deg U, \deg V < m$, neither factor is a multiple of p . The classes $[U], [V]$ are therefore nonzero, but $[U][V] = [p] = [0]$, so the quotient is not a field.

The map $c \mapsto [c]$ preserves the operations and is injective, since $p \mid (c - d)$ for constants c, d implies $c = d$. Moreover, the quotient projection preserves addition and multiplication, so $p([X]) = [p(X)] = [0]$. Finally, division by p gives representatives of degree less than m , so the first m powers of α span the quotient. A nonzero combination of these powers cannot be a multiple of p , since it has degree less than m . Hence they are independent. $\square$

The case $K = \mathbb{F}_2$, $p = X^2 + X + 1$ is the model $\mathbb{F}_4$ already constructed in Chapter 1. An irreducible polynomial of degree 128 over $\mathbb{F}_2$ gives 128 independent binary coordinates, hence 2^{128} elements. Bézout's identity was used in the proof. This is not a request to implement GCD or the extended Euclidean algorithm.

4.1.3 The fixed modulus and the choice of reduction

For this project, we choose

$$p_{128}(X) = X^{128} + \underbrace{X^7 + X^2 + X + 1}_{\rho(X)}. \quad (4.2)$$

We therefore fix $F := \mathbb{F}_2[X]/(p_{128})$, the model of $\mathbb{F}_{2^{128}}$ that we implement, and $\alpha = [X]$. In what follows, F and $\mathbb{F}_{2^{128}}$ refer to the same field once this model is chosen. The coordinates belong to the vector space $\mathbb{F}_2^{128}$.

We denote the fixed low-degree part of the modulus by $\rho(X) = X^7 + X^2 + X + 1$ and the remainder of an input C by $R(X)$.

The choice of parameter is justified by NIST SP 800-38D, §6.3, which uses this irreducible polynomial to describe the binary field with 2^{128} elements [4]. We take irreducibility as a documented property of the parameter, not as a consequence of randomized tests. NIST associates the constant coefficient with the first bit of the string, whereas here it is the least significant bit of the word. Thus a common modulus does not make this representation directly compatible with GHASH.

In the quotient, $[p_{128}] = [0]$. In characteristic two, the additive inverse of ρ is ρ , so the modulus relation becomes

$$X^{128} \equiv \rho(X) = X^7 + X^2 + X + 1 \pmod{p_{128}}. \quad (4.3)$$

This is a polynomial congruence, not an equality in $\mathbb{F}_2[X]$. The corresponding field equality is $\alpha^{128} = \alpha^7 + \alpha^2 + \alpha + 1$. Multiplying the congruence by a polynomial H lets us replace the *entire* part HX^{128} with $H\rho$, rather than one term at a time.

Proposition 12 (Folding the high part). Let $p = X^m + \rho \in \mathbb{F}_2[X]$, with $\rho \in \mathbb{F}_2[X]_{<m}$. If $C = L + HX^m$ and $L \in \mathbb{F}_2[X]_{<m}$, then

$$C \equiv L + H\rho \pmod{p}.$$

Proof. Since $X^m = p + \rho$, we have $C = L + Hp + H\rho$. Thus C and $L + H\rho$ differ by the multiple Hp of the modulus. $\square$

The benefit of this choice is now explicit. The divisor is *always the same*, and $\rho = 1 + X + X^2 + X^7$ has only four monomials. Thus $H\rho$ is a sum of four shifted copies of H , rather than a general polynomial product. On words, these operations become shifts and XORs. The maximum degree 7 of the low-degree terms also bounds what remains to be reduced after the first pass.

Long division cancels one high coefficient at a time. Folding adds the multiple $H\rho$ in one step, canceling the entire original high part. Some high coefficients may remain, and we handle them in the same way. Irreducibility guarantees the field structure, while the sparse form of the modulus makes this computation inexpensive.

Other moduli may call for other methods, including polynomial Barrett reduction based on a pre-computed reciprocal. Such preprocessing can also be reused for a modulus chosen at runtime. It is unnecessary here: we develop sparse reduction without claiming that it is the fastest solution on every processor or for every future representation.

4.2 Representation

4.2.1 The word pair and its coefficients

Let

$$C(X) = c_0(X) + c_1(X)X^{128}, \quad c_0, c_1 \in \mathbb{F}_2[X]_{<128}.$$

The word `lo` encodes the coefficients of $c_0(X)$, and `hi` those of $c_1(X)$. If

$$c_1(X) = h_0 + h_1X + \cdots + h_{127}X^{127},$$

then bit i of `hi` is h_i . In the full polynomial $C(X)$, the same coefficient multiplies X^{128+i} .

The low-degree part of the modulus is

$$\rho(X) = 1 + X + X^2 + X^7.$$

The first fold uses

$$C(X) \equiv c_0(X) + c_1(X)\rho(X) = c_0(X) + c_1(X) + c_1(X)X + c_1(X)X^2 + c_1(X)X^7.$$

This formula directly explains the shifts 0, 1, 2, and 7 applied to `hi`.

4.2.2 Generic shifts and the 128-bit boundary

The following diagrams show only the coefficients near the boundary between degrees 127 and 128. The symbols h_i are generic, with no particular numerical value chosen. The bar between 128 and 127 separates coefficients that leave the word, on the left, from those retained in it, on the right.

	129	128	127	126	125	124	123	122
$c_1(X)$	0	0	h_{127}	h_{126}	h_{125}	h_{124}	h_{123}	h_{122}
$c_1(X)X$	0	h_{127}	h_{126}	h_{125}	h_{124}	h_{123}	h_{122}	h_{121}

Figure 4.1: Multiplication by X . The coefficient h_{127} crosses the boundary and becomes the coefficient of degree 128.

	129	128	127	126	125	124	123	122
$c_1(X)X^2$	h_{127}	h_{126}	h_{125}	h_{124}	h_{123}	h_{122}	h_{121}	h_{120}

Figure 4.2: Multiplication by X^2 . The coefficients h_{127} and h_{126} move to degrees 129 and 128, respectively.

	134	133	132	131	130	129	128	127	126	125	124
$c_1(X)X^7$	h_{127}	h_{126}	h_{125}	h_{124}	h_{123}	h_{122}	h_{121}	h_{120}	h_{119}	h_{118}	h_{117}

Figure 4.3: Multiplication by X^7 . The seven coefficients $h_{127}, \dots, h_{121}$ cross the boundary.

Each shift therefore splits into a low and a high part. For the word `hi`, a left shift gives the part that remains below degree 128, while a right shift extracts and reindexes the part that crosses the boundary:

term	low contribution	reindexed high contribution
$c_1(X)$	<code>hi</code>	0
$c_1(X)X$	<code>hi << 1</code>	<code>hi >> 127</code>
$c_1(X)X^2$	<code>hi << 2</code>	<code>hi >> 126</code>
$c_1(X)X^7$	<code>hi << 7</code>	<code>hi >> 121</code>

To derive the counts 127, 126, and 121, consider a shift by k positions: the original bit at index i contributes to degree $i + k$. It leaves the word exactly when $i + k \geq 128$, that is, $i \geq 128 - k$. For $1 \leq k < 128$, the bits that leave are therefore selected by

$$\boxed{\text{hi} \gg (128 - k)}.$$

For $k = 1, 2, 7$, this gives $128 - k = 127, 126, 121$. The right shift discards the other bits and moves the first coefficient beyond the boundary to local position 0. For example, the high part of $c_1(X)X^7$ is

$$(h_{121} + h_{122}X + \cdots + h_{127}X^6)X^{128}.$$

The word `hi >> 121` encodes the polynomial in parentheses, not the entire term. The factor X^{128} remains implicit: extracting the overflow does not reduce it.

4.2.3 The first fold

Adding all contributions still gives a polynomial represented by two words:

$$C_1(X) = d_0(X) + d_1(X)X^{128}.$$

The word `fold_lo` encodes $d_0(X)$, and `fold_hi` encodes $d_1(X)$:

$$\begin{aligned} \text{fold_lo} &= \text{lo} \oplus \text{hi} \oplus (\text{hi} \ll 1) \oplus (\text{hi} \ll 2) \oplus (\text{hi} \ll 7), \\ \text{fold_hi} &= (\text{hi} \gg 127) \oplus (\text{hi} \gg 126) \oplus (\text{hi} \gg 121). \end{aligned} \tag{4.4}$$

The three high contributions are combined with XOR because they may overlap at the same degree.

4.2.4 The second fold

The high part has not yet disappeared: $C_1(X)$ still contains $d_1(X)X^{128}$. Apply the relation again:

$$X^{128} \equiv 1 + X + X^2 + X^7 \pmod{p_{128}}.$$

This gives

$$d_1(X)X^{128} \equiv d_1(X) + d_1(X)X + d_1(X)X^2 + d_1(X)X^7.$$

Hence

$$\begin{aligned} \text{reduced} &= \text{fold_lo} \oplus \text{fold_hi} \oplus (\text{fold_hi} \ll 1) \\ &\quad \oplus (\text{fold_hi} \ll 2) \oplus (\text{fold_hi} \ll 7). \end{aligned} \tag{4.5}$$

After the first pass, d_1 has degree at most 6 when nonzero, so $d_1\rho$ has degree at most $13 < 128$. The second fold cannot create a new high part, and the result is already the canonical remainder. Thus *at most* two folds suffice. For some inputs, the intermediate high part is already zero.

4.3 Implementation

In the same file `src/field/portable.rs` used in the preceding chapter, implement the free function

```
pub fn reduce_p128(lo: u128, hi: u128) -> u128
```

The input pair encodes $C(X) = c_0(X) + c_1(X)X^{128}$. The output is the word encoding its remainder modulo p_{128} . The contract accepts every pair of `u128` values, including those with bit 127 of `hi` set. No `Option`, `Result`, or check on that bit is needed.

Translate equations (4.4) and (4.5) into code. The code uses the three temporaries `fold_lo`, `fold_hi`, and `reduced`: respectively, the low part d_0 after the first fold, the reindexed high part d_1 after the first fold, and the final remainder after the second.

Compute `fold_lo` first and then `fold_hi`, without modifying `hi`. Finally, compute $d_1(X)\rho(X)$ by combining the unshifted copy of `fold_hi` with copies shifted by 1, 2, and 7 positions, add it to `fold_lo`, and return `reduced`. Simply XORing the two intermediate words does not perform reduction.

Every shift on a `u128` uses a count less than 128. Do not construct the leading term of the modulus with `1u128 << 128`: it is already represented implicitly by the split `(lo, hi)`. The integer operators `/` and `%` do not compute polynomial division and have no role in this algorithm. Operator and cast conventions follow Chapter 3 and the Rust Reference [7].

A direct translation uses nine shifts and ten XORs on `u128`: three shifts and four XORs for `fold_lo`, three shifts and two XORs for `fold_hi`, and three shifts and four XORs for `reduced`. This counts source operators, not machine instructions. No loops or data-dependent branches are needed, and auxiliary storage is fixed. These properties alone do not establish the timing behavior of the compiled program.

4.4 Verification

4.4.1 An independent coefficientwise reference

To check `reduce_p128`, we use long division in the tests as a reference independent of the two folds, prioritizing ease of verification over speed. The slow reference implementation reduces one coefficient at a time:

```
pub fn slow_reduce(lo: u128, hi: u128) -> u128.
```

The reference implementation uses a local array `v: [bool; 256]` to represent every coefficient of the wide polynomial explicitly. The array index equals the degree: `v[i]` is `true` if and only if the coefficient of X^i is one. There is no need for a new public `get_bit` function for `u128`: the primitive from Chapter 0 remains restricted to `u64`.

The first scan copies the two words into the coefficient representation. For $0 \leq i < 128$, read bit i of `lo`. For $128 \leq i < 256$, read bit $i - 128$ of `hi`. Thus `v[0], ..., v[127]` encode the low part, and `v[128], ..., v[255]` the high part of the full polynomial.

Reduction then proceeds from the highest degree downward. If `v[j]` is true with $j \geq 128$, the term X^j is present. The relation

$$X^{128} \equiv X^7 + X^2 + X + 1 \pmod{p_{128}}$$

implies

$$X^j \equiv X^{j-121} + X^{j-126} + X^{j-127} + X^{j-128} \pmod{p_{128}}.$$

The term of degree j is therefore removed, and its coefficient is added, using XOR, at the four corresponding lower positions.

This is the same alignment as in Example 4.1. Setting $D = p_{128}$, add the following polynomial to the current remainder:

$$DX^{j-128} = X^j + X^{j-121} + X^{j-126} + X^{j-127} + X^{j-128}.$$

The divisor is shifted by $j - 128$, not j : the leading term of D already has degree 128. The addition cancels X^j and preserves the class of the remainder modulo D .

Descending order is essential: reducing a high term can introduce new coefficients of degree at least 128, but always below the degree just eliminated. For example, reducing X^{255} can introduce X^{134} . Scanning 255, 254, $\dots$, 128 ensures that this new term is processed later. The assignment `v[j] = false` clears the leading term just reduced.

Implement the reference function in `tests/common/slow_reduce.rs` using the three stages described above: read the coordinates, cancel in descending order, and reconstruct the final word. The contract accepts every pair of `u128` values, not only pairs obtained as products.

The scan invariant requires that, before processing index j , every coefficient of degree greater than j be zero and the polynomial represented by the array be congruent to the input modulo p_{128} . If the coefficient of degree j is zero, nothing changes. Otherwise, add $p_{128}X^{j-128}$. This cancels the coefficient of degree j and changes only lower degrees, preserving the invariant for the next step. After step $j = 128$, the canonical representative remains.

After the scan, reconstruct the remainder word from the first 128 positions only. In this final loop, OR and XOR are equivalent: each bit is inserted once into an initially zero result. The XORs used during reduction, however, cannot be replaced with OR, since their contributions may cancel.

4.4.2 A single comparison test

The test `reduction_matches_the_slow_reference` compares `reduce_p128(lo, hi)` with `slow_reduce(lo, hi)` on 10000 random pairs.

In `tests/field_portable.rs`, the two comparison tests share these imports:

```
use binary_afft::field::portable::{clmul, clmul128, reduce_p128};
use std::rand::Rng;

#[path = "common/slow_clmul.rs"]
mod slow_clmul_ref;
#[path = "common/slow_reduce.rs"]
mod slow_reduce_ref;

use slow_clmul_ref::slow_clmul;
use slow_reduce_ref::slow_reduce;
```

The `path` attribute specifies the file containing the reference module's body, while `use` brings its function into the test's scope [7, 6].

Passing the test verifies the cases examined, not every possible pair.

The tests are in `tests/field_portable.rs`.

Review exercise. Reconstruct the algorithm from the modulus relation. Explain what `fold_lo` and `fold_hi` encode, why both are computed from the original `hi`, and why the second substitution already yields the canonical remainder.

Chapter 5

The Gf128 Type and Field Operators

The work in this chapter involves `src/field/gf128.rs`, `src/field/mod.rs`, `src/field/portable.rs`, and `tests/gf128.rs`. The word primitives are ready, so we can now construct a type distinct from `u128`, add it to the module tree, and connect Rust operators to field operations.

5.1 Algebra

The field remains

$$F = \mathbb{F}_2[X]/(p_{128}), \quad p_{128}(X) = X^{128} + X^7 + X^2 + X + 1.$$

Each element is represented by a `u128` word. For $a, b \in F$, we need only the operations already derived:

$$a + b \leftrightarrow \text{XOR of coordinates}, \quad ab \leftrightarrow \text{wide product followed by reduction},$$

$$a - b = a + b, \quad -a = a.$$

The new type gives these operations a consistent syntax. The Rust structure neither proves the field laws nor changes the representation. It simply prevents confusion between a generic word and an element of F .

5.2 Representation

5.2.1 The structure and its methods

We choose the form

```
pub struct Gf128(u128);
```

This is a *tuple struct*: `pub` exposes the type, while the inner word remains private.

Inside the module, construct a value with `Gf128(bits)`. Outside it, use `Gf128::from_u128(bits)`. The structure defines a new type, whereas `type Gf128 = u128;` would only define an alias [6, 7].

For inexpensive copying, equality comparisons, and readable test messages, add this attribute before the structure:

```
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
```

`Copy` is natural because the only data is a `u128`. The traits `PartialEq` and `Debug` also provide what `assert_eq!` requires [6, 8].

For methods defined within `impl Gf128`, `Self` denotes the type, and `self` the value received by the method. For example, a method with a second operand may have the signature:

```
fn f(self, other: Self) -> Self
```

`self: Self` is the explicit form of the same receiver. The expression `Self(bits)` constructs the result just as `Gf128(bits)` does [7, 6].

5.3 Implementation

5.3.1 The module tree: what `mod` declares

The library root, `src/lib.rs`, declares `pub mod field;`. The module body is in `src/field/mod.rs`:

```
mod gf128;
pub mod portable;

pub use gf128::Gf128;
```

The module tree and the files containing the module bodies are:

<code>crate</code>	<code>src/lib.rs</code>
<code> -- bits</code>	<code>src/bits.rs</code>
<code> -- polynomial</code>	<code>src/polynomial.rs</code>
<code>`-- field</code>	<code>src/field/mod.rs</code>
<code> -- gf128</code>	<code>src/field/gf128.rs</code>
<code>`-- portable</code>	<code>src/field/portable.rs</code>

The file `mod.rs` is not another sibling module of `gf128`: it contains the parent module, `field`. The `mod` declarations build the tree. A file does not become part of the crate merely by existing in the directory [7, 6].

`mod gf128;` keeps the module private. The declaration `pub use gf128::Gf128;` exposes its type as `binary_afft::field::Gf128`, while the inner data remains private. The module `portable` and its public functions are also accessible to integration tests [7].

5.3.2 `crate` and `super`: two ways to specify a path

Inside `gf128.rs`, the current module is `crate::field::gf128`. The keyword `crate` starts at the library root, while `super` denotes the *parent of the module in which it is written*. Here, therefore,

```
super = crate::field,    super::portable = crate::field::portable.
```

We can therefore choose between two equivalent imports [7]:

```
use crate::field::portable::mul_p128;
```

or

```
use super::portable::mul_p128;
```

We use the full path `crate::field::portable`, as in the current code.

The short name `field` instead requires an import, such as `use crate::field;`. The prefix `self::` denotes the current module, whereas the parameter `self` is the value received by the method [7].

5.3.3 use: importing a module or a function

`use` introduces a name into scope: we can import either a module or one of its functions [6, 7].

Inside `gf128.rs`, the following three forms call the same function:

Declaration in scope	Call in the method body
No import	<code>crate::field::portable::mul_p128(a, b)</code>
<code>use crate::field::portable;</code>	<code>portable::mul_p128(a, b)</code>
<code>use crate::field::portable::mul_p128;</code>	<code>mul_p128(a, b)</code>

If you import `portable` but call only functions imported separately, the module name remains unused: `unused_imports` identifies the import to remove [6, §7.4].

In the chapter on squaring, we group the two imports:

```
use crate::field::portable::{mul_p128, square_p128};
```

The standard traits require separate imports alongside these. For `Gf128`, use [8, 7]:

```
use std::ops::{Add, Mul, Neg, Sub};
```

For integration tests, we must instead account for the fact that a file in `tests/` is a separate crate: there, `crate::` denotes the test crate's root, not `src/lib.rs`. To use the library, write `use binary_afft::field::Gf128;`. The same distinction applies to examples. In local tests, `use super::*;` remains valid [6, 5].

5.3.4 The `impl Gf128` block

The `struct` defines the data, while `impl Gf128` contains constants and functions associated with the type. At this stage, add only:

Item	Required behavior
<code>ZERO: Self</code>	Element with word 0.
<code>ONE: Self</code>	Element with word 1.
<code>GENERATOR: Self</code>	The class $\alpha = [X]$, encoded by 0b10. It generates F as an algebra over $\mathbb{F}_2$, but is not claimed to generate the multiplicative group.
<code>from_u128(bits: u128) -> Self</code>	Wraps the exact input word in a <code>Gf128</code> without reduction or change of basis.
<code>to_u128(self) -> u128</code>	Returns the inner word.

Access constants and associated functions through the type, as in `Gf128::ZERO` and `Gf128::from_u128(8(bits))`. Methods receive `self`, as in `a.to_u128()` [7, 6].

Every `u128` word is a valid field representative. In tests, use `Gf128::from_u128(random_u128())` directly. Casting to `u64` would discard the high half and also give the wrong type for a function expecting `u128`: Rust does not implicitly widen an integer here [7].

5.3.5 Composing multiplication and connecting it to the type

Add to the backend:

```
pub fn mul_p128(a: u128, b: u128) -> u128
```

This function introduces no new algorithm: it simply calls `clmul128(a, b)`, obtains `(lo, hi)`, and passes the two words to `reduce_p128(lo, hi)`. The result is already the reduced word encoding the field product.

In `gf128.rs`, import `mul_p128` directly from `crate::field::portable`. To multiply two values of the type, extract `self.0` and `other.0`, call the function, and wrap the returned word with `Self(...)`. The computation follows

$$\text{operator} * \text{ on Gf128} \longrightarrow \text{mul_p128} \longrightarrow \text{clmul128} \longrightarrow \text{reduce_p128}.$$

5.3.6 Operators: a single layer of traits

We define operators through the standard traits, imported in the implementation file, without duplicate inherent methods such as `mul`.

For multiplication, use `impl Mul for Gf128 with type Output = Self;` and implement

```
fn mul(self, other: Self) -> Self
```

delegating the arithmetic to the backend already described. The result of `a * b` is a `Gf128`. The operators take their operands by value. Since `Gf128` implements `Copy`, the original values remain available to the caller [8].

For the remaining operators, `Add`, with signature `fn add(self, other: Self) -> Self`, XORs the two words and constructs the result. The operator `Sub` does the same because $a - b = a + b$, and `Neg`, with signature `fn neg(self) -> Self`, returns `self`. Tests need not import the traits to use `+`, `-`, `-a`, and `*`.

5.3.7 Implementation order

First verify the structure, visibility, constants, and conversions, then connect the operators to the backend. This separates errors in syntax and code organization from arithmetic errors. Run `cargo test` before introducing squaring or inversion.

At the end of this chapter, there is no need for `pow`, `Div`, `AddAssign`, `MulAssign`, `square`, `inverse`, or a generic `Field` trait.

5.4 Verification

5.4.1 A single minimal test of the public API

Wide multiplication and reduction have already been compared with independent reference implementations in the preceding chapters. Repeating a thousand random cases or calling `slow_reduce` would add little here: the new risk lies mainly in connecting the type, visibility, and operators incorrectly.

The test `gf128_public_api_works` in `tests/gf128.rs` verifies a conversion round trip, the words of the three constants, addition and subtraction on one pair, and the additive inverse of one element. To check the multiplication connection, `Gf128::ONE * Gf128::GENERATOR == Gf128::GENERATOR` suffices. The squaring test in the next chapter reuses the `*` operator on random inputs.

The tests are in `tests/gf128.rs`.

We have thus built a distinct type around the same `u128`, distinguished module declarations, paths, and imports, and connected standard operators to field semantics. The next chapter adds `square` and `inverse` without changing the representation.

Chapter 6

Squaring and Inversion

This chapter develops the `field/portable.rs` backend functions `spread`, `spread128`, and `square_p128`, together with the `field/gf128.rs` methods `square`, `repeated_square`, and `inverse`.

The preceding chapter constructed field multiplication: the operator `*` uses the word backend, which forms the wide polynomial product and reduces it modulo p_{128} . We could already square with `a * a`. This would be correct, but would treat two equal factors as a generic product.

Here we exploit the special structure of squaring in characteristic two. First we construct `square`, and then we reuse `square` and the operator `*` to construct `inverse`. We begin with direct spreading, although the backend currently uses the version based on shifts and masks. The transformation and the field API remain the same.

polynomial squaring $\longrightarrow$ spreading $\longrightarrow$ reduction $\longrightarrow$ `square`

`square` + `*` $\longrightarrow$ chain of powers $\longrightarrow$ `inverse`

6.1 Squaring

6.1.1 Algebra

Let $a \in F$, and let

$$A(X) = \sum_{i=0}^{127} c_i X^i, \quad c_i \in \mathbb{F}_2, \quad a = A(\alpha),$$

be its canonical representative, with $\alpha = [X]$.

Multiplying two distinct polynomials requires coefficient convolution. For a square, however,

$$A(X)^2 = \sum_i c_i^2 X^{2i} + \sum_{i < j} 2c_i c_j X^{i+j}.$$

In characteristic two, $2 = 0$. Moreover, for $c_i \in \mathbb{F}_2$, we have $c_i^2 = c_i$. Hence

$$A(X)^2 = \sum_{i=0}^{127} c_i X^{2i}. \tag{6.1}$$

The mixed terms vanish: polynomial squaring requires no new products between distinct coefficients. Each original coefficient simply moves from position i to position $2i$.

The same property underlies the Frobenius map.

Lemma 13 (Squaring and Frobenius in characteristic two). In a commutative ring of characteristic two,

$$(x + y)^2 = x^2 + y^2, \quad (xy)^2 = x^2 y^2.$$

The map

$$\varphi(x) = x^2$$

preserves addition, multiplication, and the multiplicative identity. On a finite field it is an automorphism, called the *Frobenius map*.

Proof. In the expansion of $(x + y)^2$, the term $2xy$ vanishes. The second identity follows from commutativity, and $\varphi(1) = 1$. On a field, $\varphi(x) = \varphi(y)$ implies $(x + y)^2 = 0$, hence $x = y$, so φ is injective. On a finite set, every injective self-map is also surjective. $\square$

Cancellation of the mixed terms is all we need to implement squaring. The Frobenius viewpoint becomes particularly useful for inversion: squaring s times computes

$$\varphi^s(a) = a^{2^s}.$$

Field squaring, however, requires more than polynomial squaring alone. In the quotient,

$$a^2 = [A(X)^2] = [A(X)^2 \bmod p_{128}(X)].$$

The degree of $A(X)^2$ can reach 254, whereas the canonical representative of the result must have degree less than 128. Spreading therefore constructs the *unreduced* square. The existing reduction routine converts it to the canonical field representation.

6.1.2 Representation

A word stores the polynomial coefficients. Thus equation (6.1) becomes a transformation of bit positions:

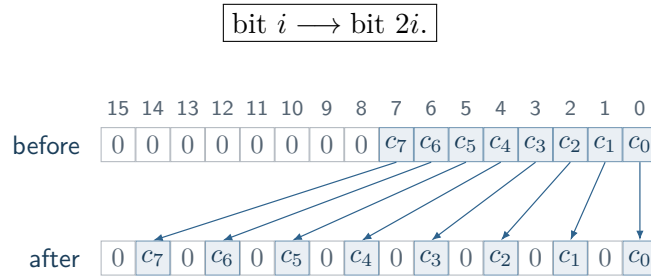


Figure 6.1: The coefficient at index i moves to index $2i$. The zeros at odd indices belong to the polynomial square before reduction.

Casting from `u64` to `u128` creates space but leaves bits in the same positions. A single shift cannot perform spreading either: `w << s` sends every index i to $i + s$, whereas the destination here is $2i$, so the displacement depends on the index.

Write

$$\text{spread} : \{0, \dots, 2^{64} - 1\} \longrightarrow \{0, \dots, 2^{128} - 1\}$$

for the transformation that, given

$$a = \sum_{i=0}^{63} \tilde{a}_i 2^i, \quad \tilde{a}_i \in \{0, 1\},$$

produces

$$\text{spread}(a) = \sum_{i=0}^{63} \tilde{a}_i 2^{2i}.$$

The highest possible output bit is bit 126.

To extend this from 64-bit to 128-bit inputs, write

$$A(X) = a_0(X) + a_1(X)X^{64}, \quad a_0, a_1 \in \mathbb{F}_2[X]_{<64}.$$

Then

$$\boxed{A(X)^2 = a_0(X)^2 + a_1(X)^2 X^{128}.} \quad (6.2)$$

Compared with the decomposition of the generic product computed by `clmul128`, the middle term vanishes:

$$p_1 = a_0 a_1 + a_1 a_0 = 0.$$

The two remaining contributions are

$$p_0 = a_0^2, \quad p_2 = a_1^2,$$

which we compute by spreading.

Let `a0` and `a1` be the 64-bit words encoding the polynomial blocks $a_0(X)$ and $a_1(X)$, respectively. The wide pair representing the square therefore has the direct form

$$\boxed{\text{lo} = \text{spread}(\text{a0}), \quad \text{hi} = \text{spread}(\text{a1}).}$$

Local bit j of `hi`, with $0 \leq j < 128$, represents the coefficient of degree $128 + j$ in the full polynomial. The factor X^{128} is already implicit in the `(lo, hi)` convention and requires no additional shift.

The calls `clmul(a0, a0)` and `spread(a0)` represent the same polynomial. Spreading therefore replaces generic carry-less multiplication when constructing the wide square, but reduction is still required.

6.1.3 Implementation

Direct spreading

To reconstruct the first algorithm, consider the signature

```
pub fn spread(a: u64) -> u128
```

and follow the mathematical definition directly: visit the 64 input bits and place the bit at index i in output position $2i$. In `src/field/portable.rs`, this loop is retained in a comment. The active implementation uses the six masking stages described below.

To reuse the commented loop, initialize the accumulator with `let mut result = 0u128`; in place of `a as u128`, which widens the input word for the masking version.

You can reuse `get_bit`. Convert the value read to `u128` before shifting, since the destination position can be as high as 126.

Let $a_i \in \mathbb{F}_2$ denote bit i of the input word. The loop invariant requires that, after processing indices $0, \dots, k-1$, the accumulator represent

$$\boxed{\sum_{i < k} a_i X^{2i}.$$

Since each position $2i$ has a single source, OR and XOR give the same result in this accumulator. In `clmul`, however, several partial products can contribute to the same coefficient, so addition must use XOR.

From the wide square to the field method

Next implement

```
pub fn spread128(a: u128) -> (u128, u128)
```

by splitting the input into two 64-bit halves and applying (6.2). The function returns the wide pair with the same convention already used by `clmul128`.

You may retain the intermediate names p_0 and p_2 to emphasize the parallel with generic multiplication. The contribution p_1 must not be computed because it is zero.

To reduce the wide square, complete the backend with

```
pub fn square_p128(a: u128) -> u128
```

by composing the wide construction with the existing reduction:

$$a \longrightarrow (\text{lo}, \text{hi}) \longrightarrow \text{reduce_p128}(\text{lo}, \text{hi}).$$

Do not introduce a separate reduction for squaring: the function `reduce_p128` depends only on its input pair, not on how that pair was obtained.

The conceptual sequence is

$$\text{spread} \longrightarrow \text{spread128} \longrightarrow \text{square_p128}.$$

To connect this computation to the field type, in `src/field/gf128.rs`, add the following method to the `impl Gf128` block:

```
pub fn square(self) -> Self
```

Bring `square_p128` into scope in the same way as `mul_p128`. The method passes the inner word to the backend and wraps the returned word in a new `Gf128` value.

Current spreading with shifts and masks

The direct version justifies the transformation. The current backend computes the same function on groups of bits, reducing the number of explicit source operations through the following six stages.

Start with 64 consecutive bits in the low half of a `u128`. First separate two 32-bit groups. Then, within each group, separate blocks of 16, then 8, 4, 2, and finally 1 bit.

In the smaller diagram below, the letters $a, b, \dots, h$ denote generic bits from most to least significant. The storage word already has 16 positions:

State	Sixteen positions, from high bit to low bit			
Widened input	0000	0000	abcd	efgh
After $s = 4$	0000	abcd	0000	efgh
After $s = 2$	00ab	00cd	00ef	00gh
After $s = 1$	0a0b	0c0d	0e0f	0g0h

Each stage simultaneously separates all blocks of the current size.

A single stage. Let U and V be adjacent s -bit blocks, with U high and V low. Locally, we want

$$[0_s \mid 0_s \mid U \mid V] \longrightarrow [0_s \mid U \mid 0_s \mid V].$$

Each box contains s bits: 0_s denotes a block of zeros and 1_s a block of ones. A plain shift would move both blocks. We therefore use an unshifted copy, a shifted copy, and a mask that retains only the two desired portions:

State or mask	Four s -bit blocks			
z	0_s	0_s	U	V
$z \ll s$	0_s	U	V	0_s
$z \mid (z \ll s)$	0_s	U	$U \vee V$	V
Mask M_s	0_s	1_s	0_s	1_s
Result	0_s	U	0_s	V

The stage transformation is therefore

$$z \leftarrow (z \vee (z \ll s)) \wedge M_s. \quad (6.3)$$

The mask clears the block $U \vee V$, where OR combines U with the unwanted copy of V produced by the shift, retaining only the shifted U and the original V . There is no overlap at retained positions, so replacing OR with XOR would give the same result after masking, provided the initial block arrangement holds. This is bit manipulation, not field addition.

The stages are

$$32 \longrightarrow 16 \longrightarrow 8 \longrightarrow 4 \longrightarrow 2 \longrightarrow 1.$$

At each step, the size of the blocks to be separated is halved. After $s = 1$, every original bit is separated from the next by a zero position.

The masks. In each window of width $2s$, the mask M_s retains the low s bits and clears the high s bits. Read from bit 127 to bit 0, it therefore alternates blocks of s zeros and s ones.

In the middle column, $(B)^{\langle r \rangle}$ denotes the concatenation of r copies of the bit string B , not a numerical power. For example, $(01)^{\langle 4 \rangle}$ is the string 01010101. Each table row thus describes the full 128-bit mask, including leading zeros:

s	Full mask, from bit 127 to bit 0	Value
32	$\underbrace{0 \dots 0}_{32} \underbrace{1 \dots 1}_{32} \underbrace{0 \dots 0}_{32} \underbrace{1 \dots 1}_{32}^{\langle 4 \rangle}$	$\frac{2^{128} - 1}{2^{32} + 1}$
16	$\left(\underbrace{0 \dots 0}_{16} \underbrace{1 \dots 1}_{16} \right)^{\langle 4 \rangle}$	$\frac{2^{128} - 1}{2^{16} + 1}$
8	$(00000000 \ 11111111)^{\langle 8 \rangle}$	$\frac{2^{128} - 1}{2^8 + 1}$
4	$(0000 \ 1111)^{\langle 16 \rangle}$	$\frac{2^{128} - 1}{2^4 + 1}$
2	$(00 \ 11)^{\langle 32 \rangle}$	$\frac{2^{128} - 1}{2^2 + 1}$
1	$(01)^{\langle 64 \rangle}$	$\frac{2^{128} - 1}{2 + 1}$

The following derivation takes place in the integers: addition, multiplication, exponentiation, and division are not operations in $\mathbb{F}_2$ or in the field F . For the stages $s \in \{1, 2, 4, 8, 16, 32\}$, $2s$ divides 128, so the word contains an integer number $64/s$ of complete blocks.

To derive the closed form, read M_s from the least significant bit. The first block of s ones has value $2^s - 1$, and the blocks of ones start at positions $0, 2s, 4s, \dots$. Thus

$$M_s = (2^s - 1) \sum_{j=0}^{64/s-1} 2^{2sj}.$$

The sum is geometric. Setting $n = 64/s$,

$$\begin{aligned} M_s &= (2^s - 1) \sum_{j=0}^{n-1} (2^{2s})^j \\ &= (2^s - 1) \frac{2^{2sn} - 1}{2^{2s} - 1} \\ &= (2^s - 1) \frac{2^{128} - 1}{2^{2s} - 1}. \end{aligned}$$

Since

$$2^{2s} - 1 = (2^s - 1)(2^s + 1),$$

we obtain

$$M_s = \frac{2^{128} - 1}{2^s + 1}, \quad s \in \{1, 2, 4, 8, 16, 32\}. \quad (6.4)$$

In the Rust representation,

$$2^{128} - 1 = \text{u128::MAX}.$$

Division is exact for these values of s , so no rounding is involved. The sum establishes the formula and must not be evaluated on each spreading call. In code, use the numerator already available as `u128::MAX` instead of attempting to compute 2^{128} in the storage type.

Computing masks at compile time. The private helper has signature

```
const fn spread_mask(i: u32) -> u128
```

The parameter `i` corresponds to stage s in the formula. Current calls use only 32, 16, 8, 4, 2, 1, but the helper does not enforce this set with an assertion. The constants `M32`, `M16`, `M8`, `M4`, `M2`, `M1` store the results.

`const fn` makes the function usable in a constant context, but it does not require every call to take place at compile time. A call in the initializer of a `const` declaration is evaluated at compile time. An ordinary call in a `let`, outside a constant context, may be optimized but provides no such guarantee [7, *Constant evaluation*].

All six masks are `pub const` in the `portable` module. The `spread` loop visits stage-mask pairs in descending order without performing divisions. Writing the same values by hand offers no advantage in this respect: in both cases, the constants are already available to the algorithm.

Stage invariant.

Proposition 14 (Bit position after a stage). After stage s , for every $0 \leq i < 64$, the original bit at index i is at position

$$\pi_s(i) = (i \bmod s) + 2s \left\lfloor \frac{i}{s} \right\rfloor.$$

All positions not occupied by these bits are zero. After the final stage $s = 1$, the bit is therefore at position $2i$.

Proof. Base case. Before the first stage, the formula holds with $s = 64$: since $0 \leq i < 64$, we have $\pi_{64}(i) = i$. The cast has left the bits at their original positions and filled the high half with zeros.

Inductive step. Assume the arrangement holds for blocks of $2s$ bits, and consider the transition to stage s . Write

$$i = 2sj + t, \quad 0 \leq t < 2s.$$

By the inductive hypothesis, before this transition the bit is at position $\pi_{2s}(i) = 4sj + t$. The corresponding window has $4s$ positions and the form $[0_s \mid 0_s \mid U \mid V]$.

If $t < s$, the bit belongs to V . The mask retains the original copy at $4sj + t$ and clears the shifted copy. If $t \geq s$, the bit belongs to U : the shifted copy at $4sj + t + s$ is retained, and the original is cleared. The new position is therefore

$$\begin{cases} 4sj + t, & 0 \leq t < s, \\ 4sj + t + s, & s \leq t < 2s. \end{cases}$$

In the first case, $i \bmod s = t$ and $\lfloor i/s \rfloor = 2j$. In the second, $i \bmod s = t - s$ and $\lfloor i/s \rfloor = 2j + 1$. Both expressions therefore equal $\pi_s(i)$.

The shift-OR-AND scheme also shows that all unwanted copies are removed. The windows are disjoint, and no retained copy leaves its window. All other positions remain zero. Induction covers all six stages and ends with $\pi_1(i) = 2i$. $\square$

The proposition proves the transformation correct. Its formula is not evaluated for each bit in the optimized implementation.

Reconstructing the current implementation. Reconstruct the six stages above and use (6.4) to derive the masks: this is the algorithm currently implemented by `spread`.

Replacing the direct loop with masks requires no changes to `spread128`, `square_p128`, or `Gf128::square`: only the implementation of the same elementary transformation changes.

The correctness condition is

$$\text{spread}_{\text{masks}}(a) = \text{spread}_{\text{direct}}(a)$$

for every input a .

Fewer visible source operations do not by themselves establish a timing advantage. Performance must be measured.

6.1.4 Verification

In `tests/gf128.rs`, the test `square_matches_multiplication_by_itself` compares specialized squaring with the existing multiplication, checking for 1000 random elements a that

$$a.\text{square}() = a * a.$$

The two paths construct the wide polynomial using different algorithms: `square` uses spreading, while `*` uses `clmul128`. They share the final reduction, which has already been compared with an independent reference implementation.

In `tests/field_portable.rs`, `test_spread` also checks that nine consecutive set bits become nine set bits separated by zeros. There is no separate randomized comparison of direct and masked spreading: the current randomized check runs through field squaring.

Random sampling makes this test a probabilistic check, not a proof for all inputs. If two functions disagree on a fraction ε of their inputs, t independent uniform samples miss every disagreement with probability $(1 - \varepsilon)^t$. A specific Schwartz–Zippel guarantee would instead require the difference between the implementations to be described by a nonzero polynomial of known degree. This assumption does not follow automatically from the code.

The construction of squaring is now complete: the backend implements `spread` with shifts and masks, and then `spread128` and `square_p128` complete the squaring operation. `Gf128::square` uses this path, and inversion will reuse its API without depending on the internal stages.

6.2 Inversion

6.2.1 Algebra

Proposition 15 (Powers of nonzero elements). In a finite field E with q elements, every $a \neq 0$ satisfies

$$a^{q-1} = 1, \quad a^{-1} = a^{q-2}.$$

Proof. Multiplication by $a \neq 0$ permutes the set $E^\times = E \setminus \{0\}$. The product $T = \prod_{x \in E^\times} x$ is nonzero, and therefore

$$T = \prod_{x \in E^\times} (ax) = a^{q-1} \prod_{x \in E^\times} x = a^{q-1} T.$$

Canceling T gives $a^{q-1} = 1$. Consequently, $a a^{q-2} = 1$, which identifies the inverse. □

In our case, $q = 2^{128}$, so

$$a^{-1} = a^{2^{128}-2}, \quad a \neq 0.$$

For every $a \neq 0$, set

$$u_k := a^{2^k-1}, \quad k \geq 1.$$

In particular,

$$u_1 = a^{2^1-1} = a.$$

These powers satisfy the fundamental Itoh–Tsuji recurrence. The original reference [3] uses normal bases. Here we apply the same algebraic identity to the fixed polynomial basis, using our squaring operation with reduction.

Proposition 16 (The Itoh–Tsuji recurrence). For $r, s \geq 1$,

$$u_{r+s} = u_r^{2^s} u_s.$$

In particular,

$$u_{2k} = u_k^{2^k} u_k.$$

Proof.

$$u_r^{2^s} u_s = a^{(2^r-1)2^s} a^{2^s-1} = a^{2^{r+s}-1} = u_{r+s}.$$

□

Since

$$2(2^{127} - 1) = 2^{128} - 2,$$

we obtain

$$a^{-1} = u_{127}^2.$$

The power $u_r^{2^s}$ is computed by applying the Frobenius map s times, that is, by s successive squarings. This is not a word shift: each square is a field operation and includes reduction modulo p_{128} .

6.2.2 Addition chain

The intermediate values u_k are ordinary **Gf128** elements, so no new representation is needed.

An *addition chain* for n is a sequence

$$1 = k_0 < k_1 < \dots < k_t = n$$

in which each new term is the sum of two earlier terms, possibly equal. Its length is t , the number of steps. We denote the minimum possible length by $\ell(n)$ [9].

To construct u_{127} , we use

$$1 \rightarrow 2 \rightarrow 3 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 15 \rightarrow 30 \rightarrow 60 \rightarrow 120 \rightarrow 127.$$

The chain has length 10 and therefore requires 10 multiplications.

In this chain, each step adds an already available index to the immediately preceding term. For consecutive indices r and $r + s$, the difference

$$s = (r + s) - r$$

has a direct meaning: it is both the number of squarings to apply to u_r and the index of the second factor u_s in

$$u_{r+s} = u_r^{2^s} u_s.$$

For the chosen chain:

Index	Sum	Element	Squarings	Multiplications
2	1 + 1	$u_2 = u_1^2 u_1$	1	1
3	2 + 1	$u_3 = u_2^2 u_1$	1	1
6	3 + 3	$u_6 = u_3^3 u_3$	3	1
7	6 + 1	$u_7 = u_6^2 u_1$	1	1
8	7 + 1	$u_8 = u_7^2 u_1$	1	1
15	8 + 7	$u_{15} = u_8^{2^7} u_7$	7	1
30	15 + 15	$u_{30} = u_{15}^{2^{15}} u_{15}$	15	1
60	30 + 30	$u_{60} = u_{30}^{2^{30}} u_{30}$	30	1
120	60 + 60	$u_{120} = u_{60}^{2^{60}} u_{60}$	60	1
127	120 + 7	$u_{127} = u_{120}^{2^7} u_7$	7	1
Final square		$a^{-1} = u_{127}^2$	1	0
Total			127	10

The number of squarings used to construct u_{127} is the sum of the differences, $127 - 1 = 126$. The final square brings the total to 127. The chain naturally splits into two parts.

In the first part,

$$1 \rightarrow 2 \rightarrow 3 \rightarrow 6 \rightarrow 7,$$

we retain $u_1 = a$. Consecutive index differences are

$$1, 1, 3, 1.$$

When the difference is 1, the second factor is u_1 . In the only other case, the difference equals the current index, giving a doubling:

$$u_6 = u_3^{2^3} u_3.$$

Once u_7 is reached, retain it and construct

$$u_8 = u_7^2 u_1.$$

From this point, u_1 does not occur in subsequent steps and needs no separate storage.

In the second part,

$$8 \rightarrow 15 \rightarrow 30 \rightarrow 60 \rightarrow 120 \rightarrow 127,$$

we retain u_7 instead. The differences are

$$7, 15, 30, 60, 7.$$

When the difference is 7, the second factor is u_7 . In the other steps, the difference equals the current index, and we apply

$$u_{2k} = u_k^{2^k} u_k.$$

The chain shows that

$$\ell(127) \leq 10.$$

Exact addition-chain tables give [10, 11]

$$\ell(127) = 10.$$

Optimality therefore concerns the number of chain steps, and hence the number of multiplications in this Itoh–Tsuji construction. It is not a lower bound on the cost of every possible inversion algorithm.

6.2.3 Implementation

In `src/field/gf128.rs`, add

```
pub fn repeated_square(self, exp: usize) -> Self
```

which applies `square` exactly `exp` times. When `exp == 0`, it returns the input. For input x , the result is

$$x^{2^{\text{exp}}}.$$

Then add

```
pub fn inverse(self) -> Self
```

for nonzero elements. The method returns `Self` directly, rather than `Option<Self>`: on entry, `assert!(self != Self::ZERO, ...)` checks the precondition and panics on zero, including in release builds.

Store the chain indices listed above in `v`, and obtain the number of squarings from consecutive differences. Initialize `result` and `uaux` with `self`. The first loop follows the first phase of the table, using the difference rules described above. Once `result` reaches u_7 , save it in `uau` and construct u_8 using `self` directly for the second factor u_1 .

The second loop retains u_7 in `uau` and updates `result` until it reaches u_{127} , following the rules of the second phase. One final `square` returns $u_{127}^2 = a^{-1}$.

No generic exponentiation routine or chain interpreter is therefore needed.

For comparison, left-to-right binary exponentiation, initialized with a at the most significant bit of $2^{128} - 2$, gives the following counts:

Method	Squarings	Multiplications
Binary exponentiation	127	126
Itoh–Tsuji, chosen chain	127	10

The savings are in multiplications. The 127 squarings remain a substantial part of inversion’s cost, so every optimization of `square` directly affects this operation.

6.2.4 Verification

In `tests/gf128.rs`, `multiplication_by_inverse_is_identity` draws 1000 random samples and checks, for every nonzero element a , that

$$a \cdot a^{-1} = 1.$$

In code, this corresponds to

```
a.inverse() * a == Gf128::ONE.
```

If the random generator returns zero, the test uses `continue` to skip the sample without replacing it, so at most 1000 checks are performed. The branch excluded from random sampling is checked separately:

```
#[test]
#[should_panic]
fn inverse_of_zero_panics() {
    Gf128::ZERO.inverse();
}
```

No separate tests of the chain’s intermediate values or a second inversion implementation are required. The tests are in `tests/gf128.rs`.

Review exercise. Reconstruct the chapter’s reasoning. Explain why polynomial squaring sends coefficient i to position $2i$, and how `spread`, `spread128`, and `reduce_p128` compose to give `square`. Explain why iterating the Frobenius map raises elements to the powers 2^s . Then explain how the Itoh–Tsujii recurrence and the chain for 127 construct the inverse, and how consecutive index differences determine the squarings and the second factor.

In the arithmetic backend for $\mathbb{F}_{2^{128}}$, `clmul128` constructs the generic wide polynomial product, while `spread128` constructs the special case of a square. The function `reduce_p128` converts both to the canonical field representative. The method `inverse` combines squares and products according to Itoh–Tsujii.

Chapter 7

Constant-Time Arithmetic

The preceding chapters developed portable arithmetic for the field

$$K = \mathbb{F}_2[X]/(P), \quad P(X) = X^{128} + X^7 + X^2 + X + 1,$$

through multiplication, squaring, and Itoh–Tsujii inversion. Algebraic correctness is not sufficient, however, when operands may depend on sensitive data. A program may always return the correct result while choosing branches, iteration counts, or memory addresses according to its input.

The starting point is commit `df2c64409183c0efa7e93d6f92610a9d40274b01` of the repository [12]. We use it to introduce a constant-time discipline in the portable backend while preserving the algebra already developed.

7.1 The constant-time requirement

The backend treats every field value and each of its coordinates as potentially sensitive, since they may depend on the witness. The polynomial P , word widths, indices of fixed-length loops, spreading masks, and the Itoh–Tsujii chain are public. The parameter `exp` in `repeated_square` is also public by contract.

Let p denote the public inputs and s the sensitive inputs. In the observation model $\mathcal{M}$, the trace $\mathcal{L}_{\mathcal{M}}(C; p, s)$ of a program C includes its control flow, the memory addresses accessed, and the effects considered observable by the model. The ideal requirement is

$$\boxed{\mathcal{L}_{\mathcal{M}}(C; p, s_0) = \mathcal{L}_{\mathcal{M}}(C; p, s_1)} \quad (7.1)$$

for all admissible sensitive inputs, with the public inputs held fixed [13].

We therefore adopt three rules for the source code:

1. sensitive data must not determine which branch is executed.
2. sensitive data must not determine how many iterations are executed.
3. sensitive data must not be used to select a memory address.

Choices that depend on sensitive data are expressed as operations on the data themselves.

A `bool` is not forbidden. The problem arises when a predicate derived from a secret is used to control execution, for example

```
if secret_predicate {  
    ...  
}
```

or in a `match`, a termination condition, or the choice of a memory index. Booleans remain appropriate in tests, examples, and public conditions. When a sensitive bit acts as a coefficient or selector in the constant-time path, we keep it as a numeric value 0/1 or use a dedicated type, such as `Choice` from the `subtle` crate [14].

These rules define a discipline at the source level. By themselves, they do not provide a universal proof about the machine code produced by every compiler on every processor. We return to this limitation at the end of the chapter.

7.2 Audit of the existing arithmetic

For each primitive, we trace the origin of branch conditions, loop bounds, and memory indices. The audit does not simply search for every `if`: a condition that depends on a public index is compatible with the contract, whereas one derived from a sensitive operand requires revision.

Component	State at the reference commit	Change
<code>get_bit</code> , <code>to_bits</code>	<code>get_bit</code> returns a <code>bool</code> .	Keep this general API. Introduce a numeric variant only for the path that needs a control word.
<code>set_bit</code> , <code>from_bits</code>	<code>set_bit</code> selects a branch according to <code>value</code> .	These remain outside the <code>Gf128</code> path. Do not use them with a sensitive selector without revising the selection.
<code>clmul</code>	The accumulator update depends on a bit of <code>b</code> .	Replace the control-flow choice with a mask-based selection.
<code>clmul128</code> , <code>mul_p128</code> , <code>Mul</code>	Call <code>clmul</code> directly or indirectly.	Revise the underlying primitive without changing Karatsuba or reduction.
<code>reduce_p128</code>	The two folds use XOR and fixed shifts.	Keep the algorithm.
<code>spread</code> , <code>spread128</code> , <code>square_p128</code>	Fixed stages, public shifts, and constant masks.	Keep the algorithms.
<code>toggle_bit</code> , <code>Add</code> , <code>Sub</code> , <code>Neg</code>	Fixed operations. The index of <code>toggle_bit</code> is public in the path considered.	Keep the algorithms.
<code>repeated_square</code> <code>inverse</code>	Performs <code>exp</code> squarings. Uses multiplication and checks whether <code>self</code> is zero with <code>assert!</code> .	Keep <code>exp</code> public by contract. Separate the computation of the candidate from the validity of the inverse.

The comparisons `index == 1` and `index == 7` in inversion depend on the public chain and may remain. The check `self != Self::ZERO` depends on the operand and must be removed from the control flow.

The audit therefore identifies two main changes: the selection within `clmul` and the treatment of zero in `inverse`. The other primitives can be reused.

7.3 Masks and branchless selection

Revising `clmul` requires choosing between a zero word and a contribution without turning the operand bit into control flow.

7.3.1 Algebra

Let $c \in \{0, 1\}$ be a selector and let x, y be words of w bits. The desired function is

$$\text{sel}(x, y, c) = \begin{cases} x, & c = 0, \\ y, & c = 1. \end{cases}$$

This definition by cases describes the mathematical function, not how it must be implemented. Define

$$M_w(c) = -c \pmod{2^w}. \quad (7.2)$$

If $c = 0$, the mask is the zero word. If $c = 1$, then $M_w(c) = 2^w - 1$, so every bit of the mask is one. It follows that

$$\text{sel}(x, y, c) = x \oplus ((x \oplus y) \wedge M_w(c)). \quad (7.3)$$

When $x = 0$,

$$\text{sel}(0, y, c) = y \wedge M_w(c). \quad (7.4)$$

7.3.2 Representation

In the backend, the coefficient at index i of the word $\mathbf{b}$ is

$$c_i = (b \gg i) \wedge 1 \in \{0, 1\}.$$

When used as an ordinary predicate, its natural representation may remain a `bool`. When it controls a constant-time selection, we represent it directly as a `u128` with value zero or one.

To preserve existing uses of `get_bit`, we introduce a second function:

```
pub fn get_bit_u128(x: u64, i: usize) -> u128
```

which computes the same coordinate in numeric form. The function `get_bit` continues to return a `bool`.

The numeric representation allows the coordinate to act as a control word without turning it into a control-flow condition. This choice does not, by itself, confer security on a particular Rust type.

For a control word $c \in \{0, 1\}$, the 128-bit mask is

$$M_{128}(c) = 0 - c \pmod{2^{128}},$$

expressed in Rust as `0u128.wrapping_sub(c)`. This is unsigned integer subtraction modulo 2^{128} , not addition in the binary field.

7.3.3 Implementation

Implement `get_bit_u128` alongside `get_bit`, preserving the latter's signature. The new function must return

$$((x \gg i) \& 1) \text{ as } u128$$

directly. The index check remains public.

For a control word `bit`, compute

$$\text{mask} = 0u128.wrapping_sub(\text{bit}).$$

When `bit` is zero, `mask` is zero. When it is one, `mask` is `u128::MAX`.

7.3.4 Verification

Correctness of the mask reduces to the two possible selector values:

$$y \wedge M_w(0) = 0, \quad y \wedge M_w(1) = y.$$

These identities will be used directly in the correctness proof for `clmul`.

7.4 Constant-time carry-less multiplication

At the reference commit, `clmul` uses a bit of `b` to decide whether to update the accumulator. The algebra remains unchanged, while the choice is moved from control flow to operations on data.

7.4.1 Algebra

For

$$A(X) = \sum_{j=0}^{63} a_j X^j, \quad B(X) = \sum_{i=0}^{63} b_i X^i,$$

the product in $\mathbb{F}_2[X]$ is

$$A(X)B(X) = \sum_{i=0}^{63} b_i A(X) X^i. \tag{7.5}$$

When $b_i = 0$, the contribution is zero. When $b_i = 1$, it is $A(X)X^i$. All 64 indices can therefore be visited along the same execution path.

7.4.2 Representation

Let a, b be the 64-bit words representing A and B . For each public index i , define

$$\tilde{b}_i = (b \gg i) \wedge 1, \quad V_i = (a \text{ as } \text{u128}) \ll i.$$

Since $\deg(AX^i) \leq 126$, each V_i fits in a `u128`. The product is represented by

$$\text{clmul}(a, b) = \bigoplus_{i=0}^{63} \left(V_i \wedge M_{128}(\tilde{b}_i) \right). \quad (7.6)$$

After the first k iterations, the accumulator represents

$$\sum_{i=0}^{k-1} b_i A(X) X^i.$$

The property holds for $k = 0$. At the next step, the mask adds $A(X)X^k$ by XOR if $b_k = 1$, or zero if $b_k = 0$. By induction, after 64 iterations the accumulator represents exactly $A(X)B(X)$.

7.4.3 Implementation

In the `clmul` loop, use `get_bit_u128` to read the coefficient of `b`. Then convert the bit into the mask M_{128} using `wrapping_sub` and apply it to the contribution before the XOR.

The required structure is

$$\begin{array}{c} \tilde{b}_i = \text{get_bit_u128}(b, i) \\ \downarrow \\ M_{128}(\tilde{b}_i) \\ \downarrow \\ ((a \text{ as } \text{u128}) \ll i) \wedge M_{128}(\tilde{b}_i). \end{array}$$

The loop still performs 64 iterations for every input. Do not add special cases for $a = 0$, $b = 0$, $a = 1$, or $b = 1$, and do not iterate only over the set bits of `b`.

The Karatsuba decomposition in `clmul128` remains unchanged. The three products

$$p_0 = a_0 b_0, \quad p_2 = a_1 b_1, \quad p_1 = (a_0 \oplus a_1)(b_0 \oplus b_1) \oplus p_0 \oplus p_2$$

are still recombined using the shifts already derived. The implementations of `reduce_p128`, `mul_p128`, `spreading`, and `squaring` also remain unchanged.

Keep `get_bit` $\rightarrow$ `bool` for general use and add `get_bit_u128` $\rightarrow$ `u128` for the constant-time path. Rewrite `clmul` with a fixed loop of 64 iterations and selection through a mask. The operand bit remains data and is not used as a branch condition.

7.4.4 Verification

The transformation must preserve the computed function. In `tests/field_portable.rs`, `clmul_match_reference` compares `clmul` and `clmul128` with the independent oracle `slow_clmul` on random inputs.

Also retain the earlier version in `tests/common/clmul_vartime.rs`:

```
use binary_afft::bits::get_bit;

pub fn clmul_vartime(a: u64, b: u64) -> u128 {
    let mut result: u128 = 0;

    for i in 0..64 {
        if get_bit(b, i as usize) {
            result ^= (a as u128) << i;
        }
    }

    result
}
```

Compare it with the new `clmul` in the test `clmul_match_vartime`. This function remains confined to tests and serves as a functional regression check. It is not used as evidence about the timing of the new implementation.

The slow oracle and the earlier version have different roles. The function `slow_clmul` checks the result against an independent procedure, while `clmul_vartime` checks that the refactoring preserves the semantics of the previous function.

7.5 Sensitive predicates

Multiplication uses a single bit as a coefficient and can represent it directly as a word. Inversion instead requires a predicate that passes through the API: we must indicate whether the returned candidate is an inverse. For this case, we use the types provided by `subtle` [14].

7.5.1 Algebra

For a word z of w bits, define

$$\text{nz}_w(z) = (z \mid (-z \bmod 2^w)) \gg (w - 1),$$

where `nz` stands for *non-zero*. The function returns

$$\text{nz}_w(z) = \begin{cases} 0, & z = 0, \\ 1, & z \neq 0. \end{cases}$$

If $z = 0$, then $-z$ is also zero. If $z \neq 0$, at least one of z and $-z \bmod 2^w$ has its most significant bit set to one. OR therefore preserves that bit, and the shift by $w - 1$ positions moves it to the least significant position.

Equality of two words reduces to checking whether their XOR is zero:

$$x = y \iff x \oplus y = 0.$$

Thus

$$\boxed{\text{eq}_w(x, y) = 1 \oplus \text{nz}_w(x \oplus y).} \tag{7.7}$$

These formulas show the algebraic principle: zero tests and equality can be represented as bits without using them directly to alter control flow. We do not reimplement them in `Gf128`. They explain the constant-time primitives already provided by `subtle`.

7.5.2 Representation

In code, we use `Choice` to represent a constant-time choice and `CtOption<T>` to associate a value with a validity bit, without introducing a new API for sensitive predicates.

This makes the rule for booleans precise. A sensitive result may be converted to `bool` only when the caller explicitly decides that the predicate is public. As long as the decision must remain within the constant-time path, it remains a `Choice` or is consumed by a constant-time primitive.

7.5.3 Implementation

Add `subtle` as a regular crate dependency, since its primitives are used by the field API as well as the tests.

Implement constant-time equality and conditional selection for `Gf128` through the trait methods

```
fn ct_eq(&self, other: &Self) -> Choice
fn conditional_select(a: &Self, b: &Self, choice: Choice) -> Self
```

delegating both operations to the internal `u128`. The field type thus exposes the primitives from `subtle` without reimplementing them.

Ordinary equality derived through `PartialEq` may remain useful in tests and when the result is public. Decisions within the sensitive path instead use `ct_eq`.

7.5.4 Verification

Check `ct_eq` on equal and unequal pairs. Also verify that `conditional_select(a,b,Choice::from(0))` returns `a` and that selecting one returns `b`. In tests, the final conversion of the predicate may be observed because the test itself is outside the library's sensitive path.

7.6 Inversion without a data-dependent exit

Once multiplication has been revised, the Itoh–Tsuji chain remains unchanged. We still need to remove the initial zero check without assigning an inverse to zero.

7.6.1 Algebra

The multiplicative inverse is defined only on

$$K^* = K \setminus \{0\}.$$

We therefore separate the computation of the candidate from its validity. Define the total function

$$J(a) = a^{2^{128}-2}.$$

For $a \neq 0$,

$$J(a) = a^{-1},$$

whereas, since the exponent is positive,

$$J(0) = 0.$$

The candidate is nonzero exactly when the input is invertible:

$$J(a) \neq 0 \iff a \neq 0.$$

Indeed, $J(0) = 0$, while for $a \neq 0$ we have $J(a) = a^{-1} \neq 0$. We therefore introduce the validity bit

$$v(a) = \begin{cases} 0, & J(a) = 0, \\ 1, & J(a) \neq 0. \end{cases}$$

The value $J(0) = 0$ is not interpreted as the inverse of zero. It is simply the candidate produced by the same chain used for every input.

With ι identifying the bits zero and one with the corresponding field elements,

$$\boxed{aJ(a) = \iota(v(a))}. \tag{7.8}$$

7.6.2 Representation

The candidate $J(a)$ is an ordinary `Gf128`, and the bit $v(a)$ is represented by `Choice`. The candidate and validity bit are returned as a `CtOption<Gf128>`, so the API does not require an early exit when the input is zero.

7.6.3 Implementation

Move the arithmetic body of the current `inverse` into an internal method

```
fn inverse_or_zero(self: Self) -> Self
```

which always executes the same chain

$$1, 2, 3, 6, 7, 8, 15, 30, 60, 120, 127.$$

With $U_j = a^{2^j-1}$, the same steps as in the preceding chapter construct U_{127} , and the final square returns

$$U_{127}^2 = a^{2^{128}-2} = J(a).$$

The 10 multiplications and 127 squarings depend on the fixed chain, not on the value of a . The comparisons `index == 1` and `index == 7` therefore depend on public data and may remain.

The method `inverse_or_zero` remains private: it is the total arithmetic core of the algorithm, not a second operation to expose through the API.

The public method becomes

```
pub fn inverse(self: Self) -> CtOption<Gf128>
```

and always performs the same steps:

1. compute the candidate $J(a)$.
2. compare the candidate with `Gf128::ZERO` through `ct_eq` and negate the `Choice` to obtain the bit `is_non_zero`.
3. construct `CtOption::new(candidate, is_non_zero)`.

By the equivalence $J(a) \neq 0 \iff a \neq 0$, this bit gives the validity of the inverse. The earlier

```
assert!(self != Self::ZERO)
```

is removed from the public inversion path.

The Itoh–Tsuji chain does not change. Only the treatment of the function’s domain changes: the arithmetic core is made total through `inverse_or_zero`, while `CtOption` carries the validity of the candidate separately.

7.6.4 Verification

The test `multiplication_by_inverse_is_identity` draws random elements, skips zero, and checks

```
a.inverse().unwrap_or(Gf128::ZERO) * a == Gf128::ONE.
```

If the `CtOption` were invalid for a nonzero element, `unwrap_or` would select `Gf128::ZERO` and the test would fail. The check therefore verifies both the candidate and its validity for nonzero inputs.

The zero case is checked separately in `inverse_of_zero_is_none`:

```
bool::from(Gf128::ZERO.inverse().is_none()) == true.
```

Here the final conversion from `Choice` to `bool` is allowed because it occurs in test code. The library does not internally turn validity into a branch that depends on the operand.

7.7 Limits of the guarantee

The constant-time discipline introduced in this chapter makes the intent of the backend explicit: in the source code, sensitive data must not determine control flow, the amount of work, or memory addresses. This avoids deliberately introducing different execution paths according to the operands.

This does not amount to a universal proof about the executable program. The compiler may transform the source, and actual latency also depends on the platform [15]. The `subtle` crate uses specific techniques to express constant-time operations in Rust, but its documentation presents these guarantees within the limitations imposed by the compiler [14].

Machine-code inspection and statistical timing tests, such as those used by `dudect` [16], may be added as further checks. They are not developed here because they depend on the toolchain and the machine and are not required to continue with the construction of the AFFT. A negative timing-test result would mean only that no leakage was detected in the configuration tested.

Review exercise. Make the portable path constant-time starting from commit `df2c644`. Keep `get_bit -> bool` for general use and introduce `get_bit_u128` for sensitive bits used as control words. Rewrite `clmul` with a fixed loop and selection through a mask. Keep Karatsuba, reduction, spreading, and the Itoh–Tsujii chain. Use `Choice`, `ConstantTimeEq`, `ConditionallySelectable`, and `CtOption` for sensitive predicates and the validity of inversion. Verify the functional correctness of the modified primitives separately.

The Cantor basis and the AFFT can then be built on a backend that treats every `Gf128` element as potentially sensitive. The future PCLMUL backend must satisfy the same contract.

Next steps

The preceding chapters complete the portable arithmetic layer for $\mathbb{F}_{2^{128}}$. Wide multiplication and specialized reduction support the field operations exposed by **Gf128**, including squaring and inversion. The next step is to construct additive domains from a Cantor basis and develop subspace polynomials. These structures will support a direct reference transform, the general AFFT, and subsequently the Dyadic AFFT.

These developments are future work: the current implementation covers field arithmetic, not the transforms. The algorithms will use the public operations of **Gf128**, without depending on the details of spreading. The hardware backend using **PCLMULQDQ** and performance work will follow the construction of the transforms. For now, the **portable** backend remains the reference.

References

AFFT references are collected for later development and do not imply that the corresponding chapters or algorithms already exist.

Algebra and finite fields

- [1] Nathan Jacobson. *Basic Algebra I*. 2nd ed. Mineola, NY: Dover Publications, 2009. ISBN: 9780486471891. URL: <https://store.doverpublications.com/products/9780486471891>.
- [2] Rudolf Lidl and Harald Niederreiter. *Introduction to Finite Fields and Their Applications*. 2nd ed. Cambridge: Cambridge University Press, 1994. ISBN: 9780521460941. DOI: [10.1017/CB09781139172769](https://doi.org/10.1017/CB09781139172769).
- [3] Toshiya Itoh and Shigeo Tsujii. “A Fast Algorithm for Computing Multiplicative Inverses in $GF(2^m)$ Using Normal Bases.” In: *Information and Computation* 78.3 (1988), pp. 171–177. DOI: [10.1016/0890-5401\(88\)90024-7](https://doi.org/10.1016/0890-5401(88)90024-7).

Standards and specifications

- [4] Morris Dworkin. *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. NIST Special Publication 800-38D. National Institute of Standards and Technology, Nov. 2007. DOI: [10.6028/NIST.SP.800-38D](https://doi.org/10.6028/NIST.SP.800-38D). URL: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38d.pdf>.

Rust and Cargo

- [5] The Rust Project. *The Cargo Book*. URL: <https://doc.rust-lang.org/cargo/> (visited on 09/19/2026).
- [6] Steve Klabnik, Carol Nichols, and Chris Krycho. *The Rust Programming Language*. The Rust Project. URL: <https://doc.rust-lang.org/book/> (visited on 09/19/2026).
- [7] The Rust Project. *The Rust Reference*. URL: <https://doc.rust-lang.org/reference/> (visited on 09/19/2026).
- [8] The Rust Project. *The Rust Standard Library*. URL: <https://doc.rust-lang.org/std/> (visited on 09/19/2026).

Addition chains

- [9] Donald E. Knuth. *The Art of Computer Programming*. Vol. 2: *Seminumerical Algorithms*. 3rd ed. §4.6.3, pp. 461–485. Reading, MA: Addison-Wesley, 1997. ISBN: 9780201896848.
- [10] Achim Flammenkamp. *Shortest Addition Chains*. URL: https://www.homes.uni-bielefeld.de/achim/addition_chain.html (visited on 09/19/2026).
- [11] OEIS Foundation Inc. *A003064: Smallest Number with Shortest Addition Chain of Length n*. The On-Line Encyclopedia of Integer Sequences. URL: <https://oeis.org/A003064> (visited on 09/19/2026).

Constant-time arithmetic and side channels

- [12] Paolo De Marinis. *binary_afft: Portable Backend at Commit df2c644*. Private repository. Sept. 19, 2026. URL: https://github.com/paolo-de-marinis/binary_afft/tree/df2c64409183c0efa7e93d6f92610a9d40274b01 (visited on 09/19/2026).
- [13] Thomas Pornin. *Constant-Time Crypto*. BearSSL. URL: <https://bearssl.org/constanttime.html> (visited on 09/19/2026).

- [14] isis agora lovecruft and Henry de Valence. *subtle: Pure-Rust Traits and Utilities for Constant-Time Cryptographic Implementations*. Version 2.6.1. URL: <https://docs.rs/subtle/2.6.1/subtle/index.html> (visited on 09/19/2026).
- [15] Moritz Schneider, Daniele Lain, Ivan Puddu, Nicolas Dutly, and Srdjan Capkun. *Breaking Bad: How Compilers Break Constant-Time Implementations*. Sept. 2, 2025. DOI: [10.48550/arXiv.2410.13489](https://doi.org/10.48550/arXiv.2410.13489). arXiv: [2410.13489v2](https://arxiv.org/abs/2410.13489v2) [cs.CR]. URL: <https://arxiv.org/abs/2410.13489v2>.
- [16] Oscar Reparaz, Josep Balasch, and Ingrid Verbauwhede. *Dude, Is My Code Constant Time?* Report 2016/1123. Cryptology ePrint Archive, 2016. URL: <https://eprint.iacr.org/2016/1123>.

Additive FFTs and binary fields: references for later development

- [17] Sian-Jheng Lin, Wei-Ho Chung, and Yunghsiang S. Han. “Novel Polynomial Basis and Its Application to Reed–Solomon Erasure Codes.” In: *2014 IEEE 55th Annual Symposium on Foundations of Computer Science*. IEEE, 2014, pp. 316–325. DOI: [10.1109/FOCS.2014.41](https://doi.org/10.1109/FOCS.2014.41). arXiv: [1404.3458v2](https://arxiv.org/abs/1404.3458v2) [cs.IT].
- [18] Sian-Jheng Lin, Tareq Y. Al-Naffouri, and Yunghsiang S. Han. “FFT Algorithm for Binary Extension Finite Fields and Its Application to Reed–Solomon Codes.” In: *IEEE Transactions on Information Theory* 62.10 (2016), pp. 5343–5358. DOI: [10.1109/TIT.2016.2600417](https://doi.org/10.1109/TIT.2016.2600417). arXiv: [1503.05761v3](https://arxiv.org/abs/1503.05761v3) [cs.IT].
- [19] Wen-Ding Li, Ming-Shing Chen, Po-Chun Kuo, Chen-Mou Cheng, and Bo-Yin Yang. “Frobenius Additive Fast Fourier Transform.” In: *Proceedings of the 2018 ACM International Symposium on Symbolic and Algebraic Computation*. Association for Computing Machinery, 2018, pp. 263–270. DOI: [10.1145/3208976.3208998](https://doi.org/10.1145/3208976.3208998). arXiv: [1802.03932v1](https://arxiv.org/abs/1802.03932v1) [cs.SC].
- [20] Ming-Shing Chen, Chen-Mou Cheng, Po-Chun Kuo, Wen-Ding Li, and Bo-Yin Yang. *Multiplying Boolean Polynomials with Frobenius Partitions in Additive Fast Fourier Transform*. Mar. 30, 2018. DOI: [10.48550/arXiv.1803.11301](https://doi.org/10.48550/arXiv.1803.11301). arXiv: [1803.11301v1](https://arxiv.org/abs/1803.11301v1) [cs.SC]. URL: <https://arxiv.org/abs/1803.11301v1>.
- [21] Nicholas Coxon. “Fast Transforms over Finite Fields of Characteristic Two.” In: *Journal of Symbolic Computation* 104 (2021), pp. 824–854. DOI: [10.1016/j.jsc.2020.10.002](https://doi.org/10.1016/j.jsc.2020.10.002). arXiv: [1807.07785v1](https://arxiv.org/abs/1807.07785v1) [cs.SC].
- [22] Susanta Samanta, Mohammadtaghi Badakhshan, and Guang Gong. *On the Additive FFT Techniques over Binary Extension Fields*. Aug. 21, 2026. DOI: [10.48550/arXiv.2608.20855](https://doi.org/10.48550/arXiv.2608.20855). arXiv: [2608.20855v1](https://arxiv.org/abs/2608.20855v1) [cs.CR]. URL: <https://arxiv.org/abs/2608.20855v1>.